NANYANG TECHNOLOGICAL UNIVERSITY

Detecting Ransomware using Deep Learning and Hardware Performance Counters

Hashil Jugjivan

Supervisor: Assoc. Prof Zhang Tianwei

College Of Computing & Data Science

2024

NANYANG TECHNOLOGICAL UNIVERSITY

CCDS24-0098

Detecting Ransomware using Deep Learning and Hardware Performance Counters

Submitted in Partial Fulfilment of the Requirements for the Degree of Bachelor of Computing (Computer Science) of Nanyang Technological University, Singapore

by

Hashil Jugjivan
U2120599F

College of Computing and Data Science
2024

Abstract

Ransomware has emerged as a significant cybersecurity threat, inflicting severe consequences including data loss, operational paralysis, reputational damage, and financial devastation. With global ransomware damages projected to reach $265 billion annually by 2031, organisations and individuals alike face increasingly sophisticated attacks that can cripple critical infrastructure and compromise sensitive information.

Traditional detection approaches using static and dynamic analysis face limitations including ineffectiveness against zero-day attacks, high system overheads, and evasion techniques employed by modern ransomware. This project proposes an innovative approach for ransomware detection utilising Hardware Performance Counters (HPCs) and deep learning techniques.

This project first analyses the limitations of existing ransomware detection methods to establish the case for hardware-based detection. Multiple deep learning architectures including Convolutional Neural Networks (CNN), hybrid CNN-RNN, Long Short-Term Memory (LSTM) networks, and Transformers are then developed and evaluated to process temporal and sequential HPC data. Furthermore, Neural Architecture Search (NAS) is applied to optimise these architectures, significantly enhancing detection accuracy while reducing model complexity.

Extensive evaluations reveal that the NAS-optimised models achieve up to 99.14% accuracy, outperforming state-of-the-art frameworks including HiPeR (98.68%) and DeepWare (98.6%). The NAS-LSTM model emerges as the most efficient solution, achieving superior performance with only 16,769 parameters.

This approach enables effective ransomware detection during the critical pre-encryption phase while maintaining minimal system overhead. The findings demonstrate that microarchitectural events captured by HPCs when analysed through optimised deep learning models, provide an effective method for ransomware detection that overcomes many limitations of traditional approaches.

Acknowledgement

The author would like to express his heartfelt appreciation to the following individuals for their continuous guidance, support, and help throughout the duration of the project.

1. Project Supervisor, Associate Professor Zhang Tianwei, for his continuous support and encouragement since the commencement of the project. The author hereby extends his sincere appreciation and gratitude for the opportunity to undertake this project.

2. PhD student Yan Xiaobei, for his continuous support and assistance throughout this project.

3. PhD student Lou Xiaoxuan, for his continuous support and assistance throughout this project.

4. Schoolmates, for their strong encouragement and support throughout his university education.

5. Family members, for their consistent support and guidance given to the author throughout his university education.

Table of Contents

List of Figures

List of Tables

1. Introduction

1.1 Background

Ransomware is a type of malware that blocks or prevents victims from accessing data on their systems and requires them to pay a ransom to access encrypted files [1]. This is a major concern for individuals and enterprises alike. Despite major security advancements in the cybersecurity landscape, ransomware remains a major current threat. According to the 2023 Cybersecurity Ventures report, ransomware is set to inflict an estimated annual cost of around USD 265 billion on its victims by 2031, making a significant surge from $42 billion in 2024 and $20 billion in 2021 [2]. A significant example was the Kaseya ransomware attack in 2021, where a zero-day vulnerability was exploited on its on-premises virtual systems administrator product to deploy ransomware to its endpoints affecting its customers and clients [3]. The need for improved ransomware detection in systems is underscored by the fact that even cybersecurity companies like Kaseya can be targeted by ransomware attacks. This highlights the urgency of being able to identify ransomware before it can cause harm.

Ransomware works in three stages, initial installation and pre-encryption, encryption of files and ransom demand [4]. During its pre-encryption stage, ransomware performs multiple operations and activities to escape various detection methods. Once encryption occurs, the damage is irreversible and thus being able to detect ransomware during its pre-encryption stage is of utmost importance.

There are two major techniques for analysing and detecting ransomware - Static and dynamic analysis [5]. Static analysis involves examining the ransomware or suspect binary file without executing it. This usually consists of generating the cryptographic signatures or cryptographic hash values associated with the suspect binary file and comparing these signatures to a database of known ransomware programs. However, static analysis requires expert knowledge of reverse engineering, and it fails to detect zero-day attacks or new ransomware programs and techniques [6].

On the other hand, dynamic analysis involves analysing the suspect binary file by executing it in an isolated environment and monitoring its activities, interactions, and effects on the system. There are three major dynamic analysis ransomware detection techniques - Threshold-based

process or file level detection, network-based detection and process behaviour-aware detection using hardware performance counters (HPC) [7]. The first technique involves detecting ransomware by monitoring file or process behaviours using thresholds for activities like input/output patterns and file entropy changes. However, this causes high system overheads due to extensive file monitoring and detects the ransomware too late. The second technique monitors communication between the victim's system and command-and-control (C&C) servers to detect unusual connections or communications. However, not all ransomware families require a C&C server to initiate encryption and waiting for communication to occur could cause the detection of ransomware to be too late. The last technique involves monitoring central processing units' HPCs to detect abnormal behaviours associated with ransomware programs. Detection using HPCs often relies on using static thresholds, which are ineffective against sophisticated ransomware. Moreover, the non-deterministic nature of HPC values produces different readings for similar processes upon each run, making detection more complicated.

Several researchers have proposed approaches to address the limitations of conventional ransomware detection methods. Sgnadurra et al. developed ElderRan, an early detection system that uses machine learning to analyse dynamic features extracted during the pre-encryption phase [8]. ElderRan achieved 96.3% accuracy in detecting zero-day ransomware samples by monitoring a combination of API calls, registry operations, and file system activities.

Researchers have begun exploring hardware-based detection methods to address the challenges of high system overheads in traditional dynamic analysis. Alam et al. demonstrated that HPCs could be leveraged to detect side-channel attacks using machine learning techniques, establishing the foundation for using microarchitectural events to identify malicious activities [9]. Building on this approach, Podolanko et al. conducted one of the first studies specifically using HPCs for ransomware detection, achieving over 90% accuracy with support vector machine (SVM) classifiers through monitoring cache behaviour and memory access patterns [6].

Most recent advancements include DeepWare [7], which transforms HPC data into image-like representations for analysis with convolutional neural networks, achieving 98.6% accuracy. Hiper [10], further improved this approach by using a feature selection algorithm to identify the

most relevant HPC metrics, resulting in 98.68% accuracy while reducing computational overhead.

This project builds upon these hardware-based approaches by using advanced deep-learning techniques and neural architecture search (NAS) to detect ransomware through HPC monitoring. While prior works have applied standard machine learning and neural network architectures to HPC data, this project leverages NAS to automatically discover optimal neural network architectures specifically tailored for ransomware detection. This approach captures and compares fluctuations in HPC values of ransomware executables with benign applications to effectively deal with the non-deterministic nature of HPC traces, with the goal of improving detection accuracy while maintaining computational efficiency. The combination of HPCs for low-overhead data collection and NAS for architecture optimization represents a novel approach to developing more effective and resource-efficient ransomware detection systems.

1.2 Objective

The objective of this project is to develop and evaluate deep learning architectures for ransomware detection using HPCs and optimise these architectures through Neural Architecture Search (NAS) to achieve high detection accuracies. The sub-objective can be further specified as follows:

1. Analysis and pre-processing of a comprehensive dataset of HPC traces from both ransomware and benign applications.
2. Identification and analysis of limitations in existing ransomware detection methods using static and dynamic analysis approaches, establishing the case for hardware-based detection.
3. Implement various deep learning architectures to assess their effectiveness for ransomware detection.
4. Application of NAS to optimise neural network architectures.
5. Conduct a comparative analysis of model performance using multiple evaluation metrics to identify the most effective architecture for ransomware detection.

1.3 Scope

The scope of this project includes the comprehensive analysis of ransomware and benign software samples, leveraging HPCs to identify distinguishable behavioural patterns. The HPC values extracted from these samples will then be processed to create the input features necessary for training deep learning models capable of addressing the non-deterministic nature of HPC traces.

The fluctuations in features from HPC traces of the ransomware and benign samples are to be extracted via an Elastic Computing Cloud (EC2) from Amazon Web Services (AWS) [11] hosting a virtual machine using VirtualBox [12]. This allows the environment in which the samples are being executed to be isolated and safe for extraction of the HPC traces.

These extracted features will then be processed and used as inputs to train a selection of deep-learning models. These models will then be optimised using NAS based on the Optuna Python library to systematically explore architectural design spaces, evaluating candidate models based on validation performance. The manually designed and NAS-optimised models will be evaluated and selected based on their accuracy and ability to effectively utilize the HPC features for ransomware detection.

This project also aims to benchmark the developed models against existing techniques to evaluate their efficiency in detecting ransomware during the critical pre-encryption phase. By focusing on the use of HPC features and deep learning techniques, the project seeks to offer a scalable and robust solution for ransomware detection.

2. Ransomware Detection

This section discusses the types of ransomware, the relevance and importance of ransomware detection and a comparison between the different existing ransomware detection techniques.

2.1 Types of Ransomware

The main goal of ransomware, as conveyed in its name, “ransom”, is to demand payment or ransom for stolen personal data and information, and stolen functionality. Over time, threat actors have diversified the way they extort money from their victims using ransomware. The table below summarises the most common types of ransomware along with a short description of them.

Ransomware	Description
Encrypting Ransomware	This type of ransomware, once executed, searches for and encrypts files of high importance to the victim. The files remain locked and encrypted until the demanded ransom is paid upon which, a decryption key or code is provided to the victim [4].
Non-encrypting Ransomware	This type of ransomware, once executed, prevents the victim access to the target machine by locking access to the device. Only upon successful ransom payment, will the target machine be unlocked [4].
Leakware	This type of ransomware, once executed, silently collects sensitive information from the victim's machine, and uses this information to blackmail the victim, by threatening to publicly release the sensitive information [4].
Scareware	This type of ransomware serves to scare users by informing them that their devices have been infected with malware when in reality they are not. Victims are then tricked into paying a fee or purchasing antivirus software to fix the issue. However, the antivirus software the scareware asks you to purchase is malicious [13].
Ransomware-as-a-Service (Raas)	Raas is a business model that sells ransomware to attackers who do not have sufficient skills or time to develop ransomware on their own. Threat actors can instead buy or rent ransomware from Raas providers [13].
Mobile Ransomware	This type of ransomware targets mobile devices and acts as a blocker instead of encrypting information, due to the ease of being able to backup and restore information to and from the cloud [4].

Table 1: Summary of Different Types of Ransomware

2.2 Ransomware Families

Countless ransomware strands have been deployed around the world with the goal of extorting money from their victims. These strands can be grouped into different families. A ransomware family is a group of ransomware variants that have similar characteristics, code signatures and functions [14]. Within a given ransomware family, threat actors develop their own techniques to gather intelligence and information and exploit vulnerabilities. The table below summarises the most notable ransomware families with a short description of them.

Families	Date Appeared	Deployment
Crypto Locker [4], [15]	2013	Compromised website and email attachments
TeslaCrypt [4]	2013	Compromised website and email attachments
Shade [4]	2015	Spam email
Petya [14], [15]	2016	Phishing
Jigsaw [4]	2016	Word document with JavaScript
WannaCry [4], [15]	2017	Samba Vulnerability
Lockbit [15],[16]	2019	Zero-day exploits
PLAY [16]	2022	Vulnerability exploitation
Akira [16]	2023	Vulnerability exploitation

Table 2: Summary of notable ransomware families

2.3 Significance of Ransomware Detection

The detection of ransomware plays a key role in modern cybersecurity, allowing for early detection of destructive malicious activity before significant loss can occur. Unlike traditional malware, ransomware specifically aims at encrypting critical information or infrastructure, often with no remorse for victims but to comply with ransom requirements. Techniques for early discovery enable both entities and individuals to respond in anticipation and therefore circumvent financial and operational consequences. As cases of ransomware become increasingly sophisticated, combining deep learning algorithms with HPCs provides a fighting chance for distinguishing between ransomware actions and normal software behaviour. If classified incorrectly, users will not be able to use the software thus causing disruptions to users.

The urgency for ransomware detection arises partly through the weaknesses in prevention methodologies, with new variants and zero-day vulnerabilities being developed in real time. Therefore, systems are powerless to claim full immunity to ransomware infections. In this case, detection is more useful than prevention, as no device can be categorised as perfectly secure. Thus, an effective detection mechanism must then be developed to mitigate the consequences of an attack. By employing methodologies of machine learning to analyse anomalies in hardware performance, it is possible to detect ransomware in its early stage - before actual information is encrypted and/or the system is locked - hence preventing irreparable loss of information.

2.4 Ransomware Analysis

This chapter discusses the concept of ransomware analysis to provide an understanding of static and dynamic ransomware analysis, its relevance and limitations.

2.4.1 Static Ransomware Analysis

Static ransomware analysis involves examining malicious software without executing the suspect binary file. This approach focuses on analysing the binary structure, code, and metadata of ransomware samples to identify their characteristics and potential threats. Static analysis is often used as an initial step in malware detection due to its non-intrusive nature. However, it has significant limitations when applied to modern ransomware, which often employs advanced evasion techniques [5].

2.4.1.1 Techniques in Static Ransomware Analysis

Static ransomware analysis involves several key techniques.

1. **File Properties and Hash Analysis:**

One of the simplest methods in static analysis involves examining the file properties such as size, format, and cryptographic hash values. Hashes like Message-Digest algorithm 5 (MD5) and Secure Hash Algorithm 1-bit (SHA-1) are generated for the ransomware sample and compared against databases of known malware signatures (e.g. VirusTotal) [17]. This helps identify known ransomware variants quickly. However, this technique is ineffective against polymorphic or metamorphic ransomware that modifies its structure or code to evade detection [18].

2. **String Extraction and Analysis:**

This technique involves extracting readable strings from the binary using tools like ‘strings’ in Linux or Sysinternals Strings (Windows) [19]. These strings may include URLs, file paths, registry keys, application-programmable interface (API) calls, ransom notes, or encryption keys. For example, strings might reveal function calls like “FindNextFileA” and “CopyFileA”, indicating file searching and copying behaviours [20]. In some cases, strings can reveal the IP addresses of controlling servers or URLs for downloading additional payloads. However, modern ransomware often employs string obfuscation techniques to hinder this analysis. This can be

addressed using advanced tools like FireEye Labs Obfuscated String Solver (FLOSS) that automatically detects, extracts and decodes obfuscated strings.

3. **Code Disassembly:**

Disassemblers like IDA Pro or Ghidra can convert binary code into assembly language for manual inspection [21]. This reveals the program's structure, control flow, and imported libraries and functions. Analysis can help identify encryption algorithms and API calls related to file encryption or network communication. However, disassembly becomes challenging when ransomware employs packing or obfuscation techniques.

4. **Metadata Analysis:**

Metadata analysis involved examining various attributes embedded within the Portable Executable (PE) file structure that can provide valuable insights into the ransomware's origin and behaviour. This analysis typically involves:

- PE Header Analysis - Examining PE headers can reveal imported libraries and entry points, indicating the ransomware's capabilities and potential behaviour [22].
- Compiler Information - Identifying the compiler used can provide insights into the development environment. This information can be obtained from specific patterns or artefacts left by different compilers in the PE structure.
- Timestamps - File creation timestamps may indicate when the ransomware was compiled.

While metadata analysis can provide valuable clues, it is important to remember that sophisticated ransomware often employs techniques to obfuscate or falsify this information.

2.4.1.2 Limitations of Static Ransomware Analysis

While static analysis provides valuable insights into ransomware characteristics, it has several limitations:

1. **Ineffectiveness against Obfuscation and Packing:**

Modern ransomware frequently uses packers and obfuscation techniques to hide its true nature, making static analysis challenging. Packers compress or encrypt the malicious code, while obfuscation methods like control flow flattening or dead-code insertion obscure the program's functionality [23]. These techniques can render traditional static analysis methods ineffective.

2. **Inability to Detect Zero-Day Ransomware:**

Static analysis heavily relies on known signatures and patterns. As a result, it struggles to identify new or modified ransomware variants that have not been previously analysed or documented. This limitation is particularly problematic given the rapid evolution of modern ransomware threats.

3. **Resource Intensity and Expertise Requirements:**

Comprehensive static analysis, especially for complex or heavily obfuscated samples, requires significant expertise in reverse engineering and can be extremely time-consuming. This makes it impractical for real-time threat detection and challenging to implement at scale.

4. **Lack of Dynamic Behaviour Insights:**

Static analysis cannot observe runtime behaviours such as process creation, file encryption activities, or network communication patterns. Many ransomware samples include conditional execution paths or environment-dependent features that only activate under specific circumstances, which static analysis cannot reveal.

These limitations highlight why static analysis, by itself is often insufficient for effective ransomware detection, especially against sophisticated and evolving threats. A comprehensive approach typically combines both static and dynamic analysis methods to provide a more robust

defence against ransomware attacks. This is where the focus of this project comes in, using HPCs in dynamic analysis to detect ransomware more effectively.

2.4.2 Dynamic Ransomware Analysis

Dynamic analysis involves executing ransomware in a safe and controlled environment, such as a sandbox or virtual machine, to observe its behaviour in real-time. This approach is particularly effective for ransomware detention, as it allows analysts to monitor the ransomware's actions, interactions with the system, and potential encryption activities. Unlike static analysis, dynamic analysis can reveal the true nature of obfuscated or polymorphic ransomware, making it a crucial tool in ransomware detection.

2.4.2.1 Techniques in Dynamic Ransomware Analysis

Dynamic ransomware analysis includes several key techniques:

1. **Sandboxing Environments:**

Dynamic analysis typically involves running suspected ransomware samples in isolated environments known as sandboxes. The suspect binary files are executed in an isolated environment using tools such as cuckoo sandbox to analyse the ransomware's behaviour and interactions with the system. These sandboxes allow security researchers to safely execute and monitor potentially malicious code while protecting the host machine from infection.

2. **System Call Monitoring:**

One of the primary dynamic analysis techniques involves monitoring system calls made by the ransomware file during execution. System calls are APIs between a user application and the operating system [24]. This provides valuable insights into how the suspect binary interacts with the operating system. It allows analysts to identify suspicious behaviours like mass file encryption, network connections, or registry modifications, often key indicators of a ransomware attack.

3. **Process Activity Analysis:**

This technique entails monitoring how ransomware communicates with the system at the process level. It involves observing and analysing the processes that are created and interacted with by a suspected ransomware sample during its execution. This approach can help identify suspicious behaviours, such as abnormal file access patterns, rapid encryption activities, or the generation of new processes that may signal malicious intent.

4. **Registry and File System Monitoring:**

This technique refers to the practice of actively observing changes made by a suspected ransomware sample to the Windows registry and file system while it is being executed. These observations are particularly valuable for detecting encryption behaviours typical of ransomware attacks, where numerous files are accessed and modified within a short timeframe.

5. **API Call Tracking:**

This technique involves monitoring and recording the specific API functions a ransomware program calls while being executed. This provides critical insights into the malicious behaviour and allows analysts to identify potential indicators of compromise (IoCs) by observing the sequence and nature of these API calls [25].

2.4.2.2 Limitations of Dynamic Ransomware Analysis

Despite their effectiveness, dynamic analysis techniques face several significant limitations that impact their practical deployment in ransomware detection systems. These limitations include:

1. **Time Complexity**

Dynamic analysis requires substantially more processing time for ransomware detection, and this poses challenges for real-time and early detection systems. This increased time complexity is due to the need to execute ransomware in a controlled environment, which can take a significant amount of time depending on the complexity of the ransomware. The time-intensive nature of dynamic analysis is particularly problematic when dealing with ransomware variants that employ rapid encryption techniques. As such, there is a great need for detection methods that can operate within extremely tight timeframes.

2. **Limited Execution Paths**

A significant limitation of dynamic analysis is its inability to explore all possible execution paths within ransomware code. Since only the paths triggered during the specific execution run are analysed, conditional behaviours may remain undetected. This inherent limitation means dynamic analysis might miss dormant malicious functionality, particularly in sophisticated ransomware variants that incorporate conditional execution paths designed to evade detection.

3. **Sandbox Evasion**

Modern ransomware increasingly employs sophisticated evasion techniques to detect sandbox environments. When these environments are identified, the ransomware can modify its behaviour to appear harmless, allowing it to evade detection. This evasive behaviour is primarily aimed at recognising and circumventing sandboxes, with fingerprinting serving as the main tactic [26]. The ransomware evaluates virtual machine drivers, specific registry keys, memory values, and performance characteristics that vary between sandboxes and native systems.

4. **Evasion through obfuscation**

Ransomware increasingly uses obfuscation techniques to hide its true nature during dynamic analysis. These techniques include code obfuscation, packing, and mutation to avoid behavioural pattern detection [27]. The ability of ransomware to alter its form while maintaining its malicious functionality presents a significant challenge for dynamic analysis approaches that rely on identifying specific behavioural patterns.

5. **Distribution of Malicious Workload**

Advanced ransomware can distribute its malicious activities across multiple cooperative processes to avoid triggering detection thresholds in dynamic analysis systems. This distribution technique operates by breaking down the encryption workload into smaller subtasks handled by separate processes that communicate with each other through various inter-process communication mechanisms [28].

2.4.3 Summary of Dynamic and Static Analysis

The table below summarises the strengths and weaknesses of current dynamic and static analysis techniques being used:

Analysis Method	Key Techniques	Strengths	Weaknesses
Static Analysis	File Properties & Hash Analysis	Quick identification of known malware and low computational requirements	Ineffective against polymorphic ransomware and requires up-to-date signature databases
	String Extraction	Reveals embedded URLs, IPs, and ransom notes and identifies potential malicious indicators	Easily defeated by string obfuscation and can produce many false positives
	Code Disassembly	Reveals program structure and logic and identifies encryption algorithms	Requires expert knowledge, time-consuming for complex samples and ineffective against packed code
	Metadata Analysis	Provides insights into origin and development and identifies compiler artefacts	Information can be easily falsified and limited correlation to malicious intent
Dynamic Analysis	Sandboxing	Safe execution environment and reveals actual runtime behaviour	Detectable by sophisticated ransomware, resource-intensive and limited execution duration
	System Call Monitoring	Captures low-level interactions and difficult for malware to evade	Generates large volumes of data and complex to analyse
	Process Activity Analysis	Identifies abnormal process behaviour and detects encryption activities	May miss distributed malicious workloads and can be challenged by obfuscated processes
	Registry & File System Monitoring	Captures critical ransomware activities and identifies encryption behaviour	High system overhead and may be too late to prevent damage

	API Call Tracking	Reveals function intent of code and identifies malicious behaviours	Bypassed with direct syscalls and is subject to API hooking evasion

Table 3: Summary of Static and Dynamic Analysis Techniques

In light of the aforementioned weaknesses of static and dynamic analysis in Table 3, researchers have begun exploring alternative detection approaches that can overcome the constraints of traditional dynamic analysis. HPCs represent a promising direction for ransomware detection, particularly for identifying previously unknown or zero-day attacks.

3. Hardware Performance Counters

This section explains hardware performance counters and their advantages over current detection methods.

3.1 Definition of HPCs

HPCs are specialised registers built into modern processors to collect microarchitectural event data during code execution. They offer insights into low-level performance metrics such as the number of executed instructions, cache misses, branch mispredictions, and CPU cycles. Their significance lies in facilitating detailed performance analysis and enabling runtime monitoring for various applications, including performance optimisation and security anomaly detection.

HPCs are part of the processor's internal performance monitoring unit (PMU). They provide a mechanism to count hardware-related events in real-time, delivering precise and granular data on how software interacts with the underlying microarchitecture.

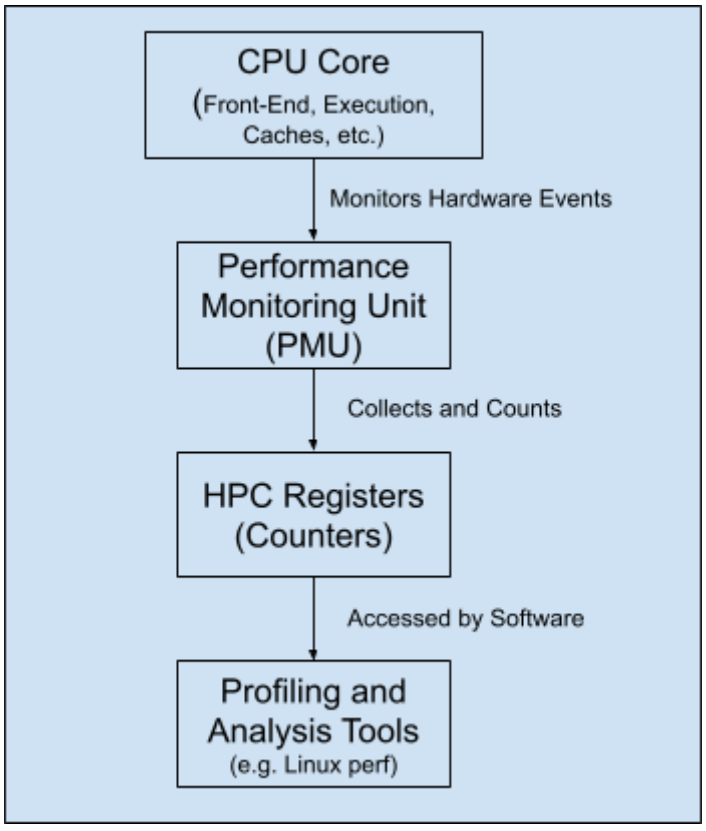

Figure 1: Simplified overview of a processor's PMU and HPC

Figure 1 provides a high-level overview of how HPC registers are integrated within a processor's PMU. As illustrated, the CPU's core components generate a range of hardware events during the program execution. The PMU monitors these events in real-time and updates the corresponding HPC registers. Profiling or analysis tools can then read these registers, enabling security researchers to capture granular, low-level metrics for performance optimisation and anomaly detection without incurring massive overhead.

For this project, we will be using event-specific counters and generic counters. Event-specific counters track predefined microarchitectural events such as the number of branch instructions, instructions retired, instructions executed, or cache accesses. Generic counters include fixed-function counters that measure a fixed set of critical events, such as total CPU cycles or total instructions executed. They operate alongside the event-specific counters and often require minimal configuration.

3.2 Hardware Performance Counters (HPCs) in Dynamic Analysis

Recent research has shown that HPCs can be leveraged for effective dynamic analysis of ransomware [6], [7],[10]. The table below summarises the advantages of using HPCs for ransomware detection.

Advantage	Explanation
Low Overhead	HPCs can be read with minimal performance impact on the system, unlike sandboxing or extensive API monitoring which can significantly slow down execution. This allows for continuous, real-time monitoring without compromising system performance.
Fine-grained Behavioural Insights	HPCs provide detailed, low-level information about program execution, including cache misses, branch predictions, and instruction counts.
Cross-platform Applicability	HPCs are available on most modern processors, making this approach applicable across various hardware architectures and operating systems.
Resistance to Obfuscation	While malware can employ various techniques to obfuscate its code or API calls, it cannot easily hide its fundamental impact on hardware performance [28].

Real-time Detection Capability	The continuous stream of HPC data allows for real-time analysis and potentially faster detection using machine learning [29].

Table 4: Summary of the advantages of using HPCs

3.3 Rationale for HPC-Based Detection Over Alternative Methods

While traditional dynamic analysis techniques such as API monitoring and sandboxing remain useful, HPCs offer unique advantages for ransomware detection over these existing methods. The table below summarises how the advantages of HPCs overcome the limitations of existing dynamic analysis detection methods.

Method	Limitations	HPC Advantages
Sandboxing	High overhead, detectable by ransomware, limited to controlled environments [7].	Minimal overhead, transparent to ransomware [7].
API Call Analysis	Evadable via direct syscalls or API hooking bypass.	Hardware-level visibility, resistant to code obfuscation.
Network Traffic	Delayed detection (post-encryption C2 communication).	Early detection (3-4 seconds pre-encryption)
Memory Analysis	Resource-intensive and requires full memory snapshots.	Lightweight, continuous monitoring via CPU registers.

Table 5: Summary of how HPCs overcome existing dynamic analysis methods

By leveraging HPCs for ransomware detection, it is possible to overcome many limitations of traditional dynamic analysis methods while gaining unique insights into program behaviour at the hardware level.

4. Deep Learning

This section discusses fundamental deep-learning techniques which will be utilised in the implementation of the model architectures for ransomware detection. These concepts form the basis for understanding the selected neural networks implemented in this project. Due to the varied characteristics and temporal nature of the data extracted from HPCs, specific deep learning architectures have been adopted to effectively capture and interpret the intricate patterns and dependencies in the data.

4.1 Fundamentals of Deep Learning

Deep learning is a subset of machine learning that makes use of artificial neural networks with multiple layers to extract higher-level features from raw input data progressively. Unlike traditional machine learning approaches that require manual feature engineering, deep learning algorithms can automatically discover intricate structures in high-dimensional data through a hierarchical learning process [30]. This capability makes deep learning particularly suitable for complex pattern recognition tasks such as distinguishing between benign applications and ransomware, based on subtle differences in their hardware performance characteristics.

4.1.1 Artificial Neural Networks

The foundations of deep learning are built upon artificial neural networks (ANN), which draw inspiration from biological neural networks in the human brain. ANNs consist of interconnected nodes, commonly referred to as artificial neurons. These neurons are the basic computational units present in ANNs and are considered to be the basic building blocks of deep learning techniques. The artificial neurons are organised in layers - an input layer, one or more hidden layers, and an output layer. Each connection between the neurons carries a weight that determines the strength of influence one neuron has on another thus affecting the final output of the neural network.

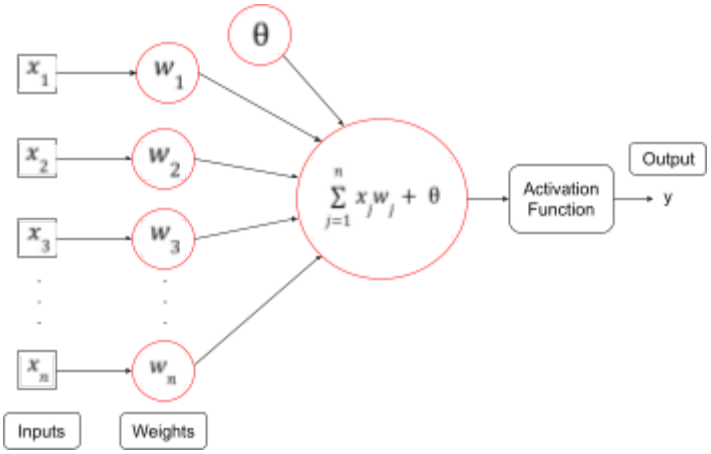

Figure 2: Illustration of an Artificial Neuron

In an artificial neuron, as illustrated in Figure 2, each artificial neuron receives multiple inputs ($x_1 x_2, x_3, \ldots x_n$), each associated with a corresponding weight ($w_1 w_2, w_3, \ldots w_n$). The inputs are multiplied by their weights and summed along with a bias (θ). The summed value is then passed through an activation function, which introduces non-linearity into the neuron's output, enabling the network to learn complex patterns from data. The neuron's output (y) then serves as an input for subsequent neurons in deeper layers of the neural network. In the context of ransomware detection using HPCs, the inputs would correspond to different hardware performance metrics, while the output would indicate the probability of the sample being ransomware.

4.1.2 Activation Function

Activation functions are mathematical operations that determine whether a neuron in a neural network should be activated based on its weighted sum of inputs it received from the preceding neurons. They introduce non-linearity into neutral networks, enabling them to learn complex patterns and relationships in data that would not be possible with purely linear transformations [31].

The fundamental purpose of activation functions is threefold. First, they introduce non-linearity, allowing neural networks to approximate complex functions and learn from high-dimensional data [31]. Second, they help control the flow of information through the network by determining which neurons should be activated based on their inputs. Third, they assist in the

backpropagation process by providing differentiable functions that enable gradient-based optimisation methods to adjust network weights effectively.

In this project, three activation functions were used - Rectified Linear Unit (ReLU) activation function, Sigmoid activation function and the Hyperbolic Tangent (tanh) activation function.

The ReLU activation function is defined as follows [32], where x represents the output of the neural network layer:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

The objective of the ReLU activation function is to determine whether a neuron should be activated by outputting the input directly if it is positive or outputting zero otherwise [32]. This determines whether the output of a neuron should be transferred over to the next neural network.

The Sigmoid activation function also commonly referred to as the logistic function is defined as follows [33], where x represents the output of the neural network layer:

$$\text{Sigmoid Function: } \sigma\,(x) \;=\; \frac{1.0}{1.0 + e^{-x}}$$

The Sigmoid function determines whether a neuron should be activated by transforming the input into a value between 0 and 1, making it particularly suitable for binary classification problems, as it provides a probabilistic interpretation of the neuron's output [33].

Lastly, the Hyperbolic Tangent activation function is defined as follows [33], where x represents the output of the neural network layer:

$$tanh(x) \;=\; \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

The tanh function determines whether a neuron should be activated by transforming the input into a value between -1 and 1. This zero-centred nature of the tanh function leads to faster convergence during training compared to the sigmoid function [33].

4.2 Deep Neural Networks

Artificial neurons become considerably more powerful when organised into deep neural networks (DNNs), composed of multiple interconnected layers. Unlike simpler ANNs that only contain a few layers, DNNs are characterised by having numerous hidden layers between the input and output layers [34], as illustrated in Figure 3 below. The architecture of a typical DNN consists of three fundamental types of layers:

1. **Input Layer:** This is the first layer that receives raw input data.
2. **Hidden Layers:** These intermediate layers perform computations that transform the input data into meaningful representations. The number and size of hidden layers determine the complexity and capacity of the network.
3. **Output Layer:** This is the final layer that produces predictions or classifications based on the processed data from the hidden layers.

Each hidden layer progressively abstracts and refines features extracted from the preceding layers, enabling the network to learn intricate patterns and relationships in the data. An illustration of a four-layer DNN is shown below where, W and b represent the weights and biases associated with the hidden layer respectively, and V and c represent the weights and biases associated with the output layer respectively :

$$Input\ x\ =\ (x_1 x_2,\ldots, x_n)^T$$

$$Hidden\ Layer\ Output,\ h\ =\ (h_1 h_2,\ldots, h_m)^T$$

$$Output\ y\ =\ (y_1 y_2, y_3,\ldots y_m)^T$$

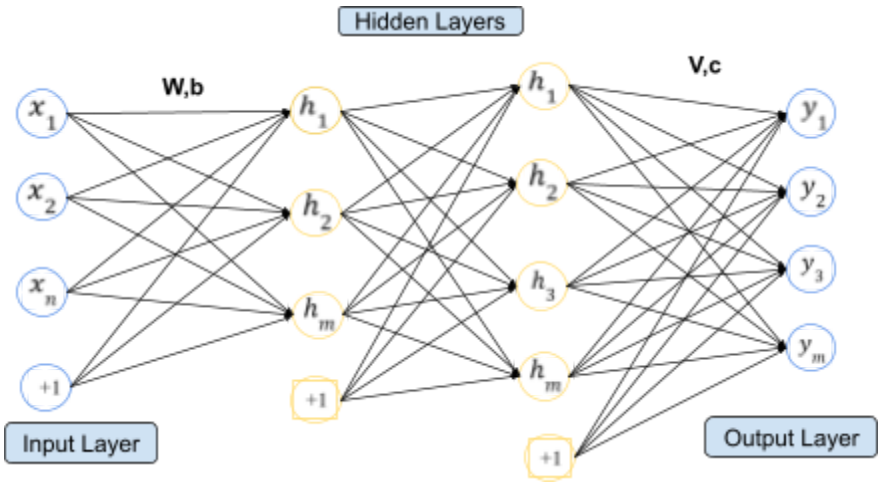

Figure 3: Example of an n-layer DNN

As mentioned in section 4.1.2, activation functions, play an important role in enabling DNNs to learn non-linear mappings between inputs and outputs. Without activation functions, each layer would perform only linear transformations, limiting the network's ability to capture complex patterns.

4.2.1 Training Deep Neural Networks

The training process is fundamental to enabling DNNs to learn complex patterns and relationships from data. Training a DNN involves systematically adjusting its parameters such as the weights and biases to minimise the cost function. A cost function is an equation used to measure the difference between predicted outputs and actual target values [35]. The cost function is minimised when the algorithm's predicted value is as close as possible to the target value. This enables the network to gradually improve its performance on the specified task. The process of training a DNN consists of:

1. **Forward Propagation:** Input data flows through the network from the input layer, through hidden layers, to the output layer. During this process, each neuron computes a weighted sum of its inputs, adds a bias term, and passes the result through an activation function, generating predictions [36].

2. **Loss Calculation:** After forward propagation produces a prediction, the network calculates how far this prediction deviates from the actual target value using a loss function [37].
3. **Backward Propagation:** After forward propagation and loss calculation, backpropagation works by computing gradients of the loss functions with respect to each weight and bias in the network propagating these gradients backwards from the output layer to the input layer [38].
4. **Weight Updates:** Weights and biases are adjusted iteratively based on gradients to minimise loss.

The iterative nature of deep learning involves training for multiple epochs which are complete passes through the training dataset, with each epoch consisting of forward propagation, loss calculation, backpropagation, and parameter updates. This process continues until the model converges to a satisfactory solution or meets predefined stopping criteria.

4.3 Convolutional Neural Networks

Building upon the fundamental concepts of DNNs, Convolutional Neural Networks (CNNs) represent a specialised class of neural networks specifically designed to process structured grid data, particularly effective for visual data and time-series analysis. Unlike traditional fully connected networks where every neuron connects to all neurons in adjacent layers, CNNs leverage the principles of local connectivity and weight sharing, enabling them to extract spatial and temporal patterns efficiently. These properties make CNNs particularly suitable for applications like ransomware detection, where HPCs produce structured data with intricate dependencies. An example of a typical CNN is shown below:

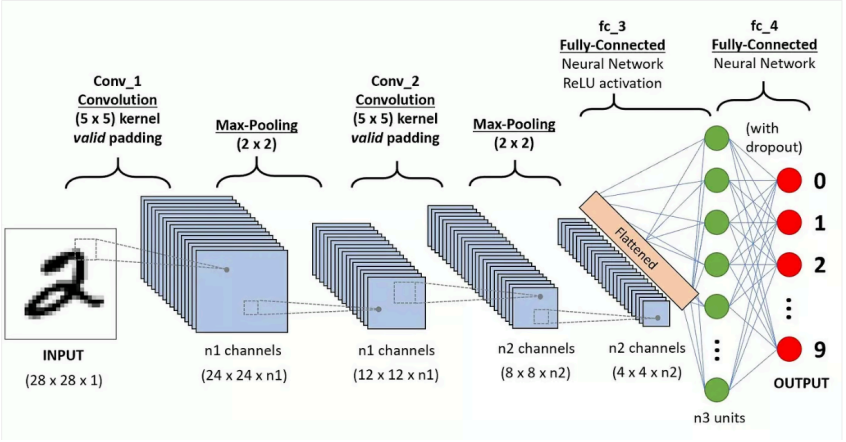

Figure 4: Typical Architecture of a CNN [39]

Figure 4 illustrates the typical architecture of a CNN which consists of three main types of layers known as convolutional layers, pooling layers and fully connected layers.

4.3.1 Convolutional Layers

The convolutional layer forms the core building block of a CNN and is responsible for feature extraction. In this layer, a set of learnable filters, also known as kernels, slides across the input data, performing convolution operations to produce feature maps [39]. Each filer is designed to detect specific patterns or features in the input data, such as edges, textures, or more complex patterns depending on the depth of the network. The key characteristics of convolutional layers include the following:

1. **Local Connectivity:** Each neuron in a CNN connects to only a small region of the input volume, known as the receptive field. This reduces the number of parameters compared to fully connected networks, enabling deeper architectures without overfitting
2. **Weight Sharing:** The same filter weights are applied across the entire input, allowing to network to detect features regardless of their position in the input.
3. **Multiple Filters:** Each convolutional layer typically applies multiple filters to the input, each learning to detect different features. This results in multiple feature maps that collectively represent different aspects of the input data.

4.3.2 Pooling Layers

Pooling layers follow convolutional layers to down-sample the feature maps produced by the convolutional layers and reduce their spatial dimensions further. Similar to the convolutional layer, the pooling layer slides a weightless filter across the entire input and applies an aggregation function to the values within that region to the output array [40]. Moreover, pooling helps mitigate overfitting by reducing the number of parameters in subsequent layers while preserving essential features. There are two main types of pooling:

1. **Max Pooling:** Selects the maximum value from each local region of the feature map. This operation retains the most prominent features while discarding less significant details.
2. **Average Pooling:** Computes the average value of each local region, providing a smoother representation of features.

For HPC-based ransomware detection, pooling layers help in identifying temporal patterns in HPC data by focusing on significant variations while being robust to minor fluctuations due to the non-deterministic nature of hardware events.

4.3.3 Fully Connected Layers

After several convolutional and pooling layers extract hierarchical features, one or more fully connected layers interpret these features for classification or regression tasks. Each neuron in a fully connected layer connects to every neuron in the previous layer, enabling it to combine information extracted from all extracted features. The output of the finally fully connected layer is typically passed through an activation function appropriate for the task at hand. For ransomware detection, fully connected layers map extracted temporal and spatial patterns and use a sigmoid activation function to output probabilities indicating whether an application is ransomware or benign.

4.3.4 Dropout Layers in CNNs

Dropout is a regularisation technique used in CNNs to reduce overfitting, which occurs when the model learns the training data too well and fails to generalise to unseen data [41]. Dropout mitigates this issue by randomly deactivating a fraction of neurons and their connections during training, forcing the network to learn more robust and generalised features. During training, dropout randomly drops neurons in a layer by setting their activation values to zero with a specific probability known as the dropout rate. This prevents the dropped neurons from contributing to the next layer and ensures their weights are not updated. This effectively created a 'thinned' network for each training pass where only a subset of the neurons are active [41].

The key idea is to prevent neurons from becoming overly dependent on one another, thereby encouraging the network to learn redundant and independent features. In HPC-based ransomware detection, dropout layers are particularly useful in fully connected layers where dense connections increase the risk of overfitting thereby enhancing the model's ability to generalise across diverse ransomware behaviours.

4.4 Recurrent Neural Networks

Building on the concepts of DNNs, Recurrent Neural Networks (RNNs) represent a specialised class of neural networks designed to process sequential data. Unlike traditional feedforward networks, which process data in one direction from input to output, RNNs incorporate a feedback mechanism that allows them to maintain a form of memory by retaining information about previous inputs [42]. This makes RNNs particularly useful for tasks involving temporal dependencies, such as the analysis of time-series data from HPCs for ransomware detection.

Recurrent neurons are the fundamental processing units in RNNs. The structure of a recurrent unit is shown below in Figure 5 where h represents the unit's hidden state, X represents the input at the current time step, L represents the output of the recurrent neuron and the current time step, u represents the weight associated with the input X, v represents the weight of the feedback loop, and w represents the weight associated with the connection from the hidden state h to the output L:

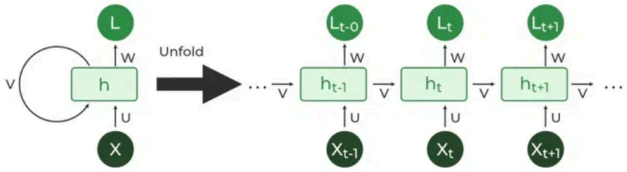

Figure 5: Architecture of a Recurrent Neuron and an RNN [42]

The defining feature of recurrent units is their ability to process sequences by maintaining a hidden state that captures information about previous time steps. At each time step as shown in Figure 5, the network takes the current input X_t, weighted by u, and the hidden state from the previous time step h_{t-1}, weighted by v, to update the current hidden state h_t. This updated hidden state is then used to compute the output L_t, influenced by weight w. The new hidden state h_t is passed forward to the next time step. This feedback mechanism enables RNNs to model temporal patterns and dependencies in sequential data.

4.5 Long Short-Term Memory

Building upon the concept of RNNs, Long Short-Term Memory (LSTM) networks are an enhanced type of RNN designed to address the limitations of standard RNNs, concerning their inability to learn long-term dependencies due to the vanishing gradient problem. LSTMs achieve this by incorporating a unique gating mechanism that enables them to retain information over extended time intervals, making them particularly effective for tasks involving long-term dependencies in sequential data [43], such as ransomware detection using HPCs.

The key innovation in LSTMs lies in their memory cell and gating mechanisms, which regulate the flow of information through the network. These gates determine what information is to be kept, discarded or output at each time step, preventing irrelevant data from accumulating over time. The gating mechanisms are implemented using a sigmoid activation function which produces outputs between 0 and 1, effectively acting as on or off switches. An LSTM cell consists of the following components:

1. **Input Gate:** This gate determines what new information should be added to the cell state. The input gate consists of a sigmoid layer and a tanh layer. The sigmoid layer decides which values to update, and the tanh layer creates a vector of candidate values for updating the cell state.
2. **Forget Gate:** This gate decides which information from the previous cell state should be discarded or "forgotten". If the output for a particular cell state is 0, the piece of information is forgotten while if the output is 1, the information is kept for future use.
3. **Cell State Update:** This feature involves combining the output of the forget gate and input gate to update the cell's state.
4. **Output Gate:** This gate is responsible for determining what information from the current cell state should be output at the current time step.

Unlike traditional RNNs, which overwrite their hidden state at each time step, LSTMs maintain a separate cell state which provides long-term memory functionality. The gating mechanism dynamically adjusts how information flows through the network, allowing LSTMs to focus on relevant inputs while ignoring noise. This enables effective learning of long-term dependencies making LSTMs highly useful for analysing sequential HPC data in ransomware detection.

4.6 Transformers

Transformers are a special type of deep learning architecture that makes use of self-attention mechanisms to process and generate sequential data in parallel compared to architectures like RNNs and LSTMs that process sequential data in order [44]. This enables transformers to efficiently model long-range dependencies in sequential data, making them particularly useful for tasks like ransomware detection using sequential HPC data.

Transformers are built around the encoder-decoder framework. The encoder processes the input sequence and generates a contextualised representation of the data that can be used by the decoder to generate the output. Before the input can pass through the encoder-decoder framework, the input data needs to be processed. This involves converting the raw text input data into a format that is understandable by the transformer model, using tokenization and embedding. Tokenization splits the input text into smaller manageable units called tokens.

Embedding consists of converting the tokens into fixed-sized vectors using an embedded layer. To provide information about the position of the tokens within the sequence, positional encodings are added to the embeddings. After which, the input sequence is passed through the encoder-decoder framework.

The encoder consists of the following sub-layers:

1. **Self-Attention Mechanism:** This sub-layer is the core innovation of transformers and it allows the model to capture dependencies regardless of the distance within sequences by weighing the importance of different parts of the input sequence [44].
2. **Feed-Forward Neural Network:** This sub-layer generates a refined representation by processing the output of the self-attention mechanism using a ReLU activation function [44].
3. **Layer Normalisation and Residual Connections:** Stabilises training by preserving input information and improving gradient flow.

The decoder, on the other hand, uses the representations generated by the encoder and previously generated tokens of the output to generate the output sequence. The decoder consists of the following sub-layers:

1. **Masked Self-Attention Mechanism:** This mechanism is similar to the encoder's but prevents the decoder from attending to future tokens in the output sequence using a mask [44].
2. **Encoder-Decoder Attention Mechanism:** Allows the decoder to focus on only the relevant portions of the encoder’s representation [44].
3. **Feed-Forward Neural Network:** This sub-layer generates a refined representation by processing the output of the encoder-decoder and masked self-attention mechanism using a ReLU activation function [44].

4.7 Neural Architecture Search

Neural Architecture Search (NAS) is an automated process for designing optimal neural network architectures that are tailored towards specific tasks and falls within the realm of automated machine learning (AutoML) [45]. Traditionally, neural networks were manually designed by human experts however this manual process is time-consuming, error-prone, and limited by human intuition. NAS has overcome these limitations by leveraging machine learning techniques to explore the vast space of possible architectures systematically. This makes NAS particularly valuable for complex applications such as ransomware detecting using HPCs, where identifying the best-performing architecture can significantly enhance detecting accuracy.

At its core, NAS operates as a search algorithm that explores a predefined search space of possible network topologies. The search space includes various architectural components, such as convolutional layers, recurrent layers, pooling operations, and fully connected layers, along with their respective hyperparameters such as batch sizes, filter sizes, number of neurons, activation functions etc. The search algorithm evaluates the proposed candidate architectures based on their performance on validation data and iteratively refines the search to identify the optimal algorithm. NAS can be broken down into three main components: search space, search strategy or algorithm and evaluation strategy. These components can be used in several ways to maximise the search for the optimal neural network architecture.

4.7.1 NAS Search Space

The search space in NAS defines the range of neural network architectures that the algorithm explores to find an optimal model architecture. The search space includes architectural elements such as the types of layers (convolutional, pooling, recurrent), their connectivity patterns, the number of neurons or filters, and hyperparameters such as kernel sizes. The search space significantly influences the effectiveness and efficiency of NAS, and it defines both the complexity and diversity of candidate architectures.

A well-defined search space balances complexity and practicality. Overly large search spaces may contain more potential architectures but can also hinder effective exploration due to increased computational costs and complexity [46]. However, overly constrained search spaces might limit the discovery of novel and high-performing architectures.

4.7.2 NAS Search Strategy

The search strategy defines how NAS algorithms systematically explore the defined search space to identify optimal neural network architectures. Effective search strategies balance exploration (discovering novel architectures) with exploitation (focusing on promising candidates). There are several prominent NAS search algorithms:

1. **Reinforcement Learning (RL):** RL-based NAS methods model architecture selection as a sequential decision-making process, where an agent learns to select network configurations based on performance feedback from validation accuracy or other metrics [47].
2. **Evolutionary Algorithms (EAs):** EAs iteratively generate populations of candidate architectures, evaluate their performance, and select top performers to produce subsequent generations through mutation and crossover operations [47].
3. **Bayesian Optimisation:** This approach builds probabilistic models to predict architecture performance based on previously evaluated configurations guiding searches towards promising regions in the architecture space efficiently [47].
4. **Gradient-Based Methods:** This algorithm represents architectural parameters continuously and optimises them using gradient descent techniques [47].

Each strategy has its strengths and limitations and choosing an appropriate algorithm depends on available computing resources and speed of convergence. For example, RL has achieved state-of-the-art results but requires extensive computational resources, while EA offer robustness but may converge slowly.

4.7.3 NAS Evaluation Strategy

Evaluating candidate architectures accurately but efficiently is crucial for NAS due to the large number of possible configurations within the search space. The evaluation strategy measures or estimates each potential candidate architecture's performance on a given task thereby providing feedback that guides subsequent searches. Common evaluation strategies include the following:

1. **Full Training and Validation:** This is the most direct approach that involves training each architecture fully on the training dataset and evaluating its validation accuracy. However, this method is computationally expensive and limits exploration capacity [47].
2. **Proxy Metrics:** To reduce computational costs, proxy metrics evaluate architectures using simplified criteria such as training for fewer epochs or using smaller subsets of data. This facilitates rapid estimations of performance while maintaining reasonable accuracy [47].
3. **Weight Sharing Methods:** This approach trains a single large supernet consisting of multiple candidate architectures simultaneously by sharing weights among them. Individual architectures are then evaluated by extracting subnetworks from the supernet without retraining them from scratch every time.

4.7.4 Benefits of NAS in HPC-Based Ransomware Detection

Utilizing NAS to determine the best neural network designs for the DNNs employed in this project, including CNNs, RNNs, LSTMs, and transformers, presents the following benefits:

1. **Automated Optimisation:** NAS systematically identifies optimal neural network designs tailored specifically for distinguishing ransomware behaviour patterns in HPC metrics without time-consuming manual interventions.
2. **Improve Performance:** By exploring diverse architectural possibilities beyond human intuition limitations, NAS-derives models that often outperform manually designed networks in accuracy and robustness [48].
3. **Reduced Human Effort:** Automating architecture design reduces human bias, and the effort involved in manual experimentation and exploration.
4. **Adaptability:** NAS can adaptively discover suitable architectures tailored specifically towards varying ransomware families or evolving ransomware behaviours.

Through these advantages, NAS significantly enhances deep learning model development efficiency while ensuring high-performance detection capabilities suitable for real-time cybersecurity applications.

5. Implementation

This section details the implementation of the Ransomware Detection System using Deep Learning Techniques and Hardware Performance Counters. It encompasses dataset collection, pre-processing, model development, and training.

The full code implementation can be found at the following GitHub link: https://github.com/Hashil-Jugjivan/Ransomware-Detection-using-Hardware-Performance-Counters-and-Deep-Learning.git.

5.1 Deep Learning Environment Setup

The designing, training and testing of the ransomware detection models were conducted in a Windows 11 operating system with an NVIDIA GeForce RTX 3070Ti with 16Gb of Virtual RAM. The software environment was set up using Python version 3.11.3 which is widely used for deep learning applications due to its extensive library support and ease of use. Several Python libraries were employed to implement the models and pre-processing techniques including Scikit-learn, PyTorch, TensorFlow, Optuna and Matplotlib.

Scikit-learn is a Python library that is particularly useful for machine learning use cases. The library contains a variety of efficient tools for machine learning and statistical modelling. It was employed for data pre-processing tasks such as normalisation and splitting of datasets into training, validation and testing sets.

PyTorch was used for implementing advanced architectures such as transformers due to its dynamic computation graph capabilities and flexibility in customising model components. PyTorch also provides seamless integrations with GPU acceleration.

TensorFlow is an open-source machine learning library that was used as the primary framework for developing deep learning models such as CNNs, LSTMs, hybrid CNN-RNNs and Transformers. TensorFlow provides high-level APIs for building, training, and evaluating neural networks while supporting GPU acceleration for faster computations.

Optuna is an open-source hyperparameter optimisation framework designed to automate the search for optimal configurations in machine learning and deep learning models. It provides a flexible and efficient approach to exploring large search spaces, leveraging optimisation techniques such as Bayesian Optimisation. Optuna allows dynamic definitions of search spaces, supports distributed optimisations and integrates seamlessly with deep learning frameworks like PyTorch.

Matplotlib was employed to visualise training progress through loss curves and confusion matrices. These visualisations helped monitor model performance during training and evaluate its accuracy on test data.

5.2 Dataset

This section discusses the dataset used for ransomware detection, including its collection methodology, structure, pre-processing techniques, and partitioning strategy for deep learning model development.

5.2.1 Dataset Collection

Due to the lack of hardware resources and the inherent risks associated with executing ransomware samples on a local device, a publicly available dataset [49] was utilised for the project. The dataset, hosted on GitHub, provides HPC traces for both malicious and benign samples.

The dataset was collected using an Elastic Computing Cloud (EC2) instance provided by Amazon Web Services (AWS) [11], which hosted a virtual machine (VM) configured with VirtualBox [12]. The VM environment was equipped with Microsoft Windows 7, along with all necessary drivers, runtime environments, and an internet connection restricted by a firewall to allow communication only with specific IP addresses.

HPC traces were extracted using the PerfExtract tool, which monitors microarchitectural events during program execution. The dataset includes traces from 23 different HPC counters sampled at intervals of 0.5 seconds over a duration of 30 seconds for each sample. These counters capture

critical metrics such as cache misses, branch predictions, CPU cycles, and instructions retired, providing detailed insights into program behaviour.

The dataset consists of 414 malicious samples obtained from VirusTotal [50] and 288 benign samples which are non-malicious applications representing typical software and program behaviour.

5.2.1.1 Elastic Cloud Computing

Amazon EC2 is a cloud computing service provided by AWS that offers scalable and resizable virtual servers, known as instances, to meet diverse computational needs. EC2 allows users to use virtual machines on demand, enabling them to develop, deploy, and manage applications without the need for physical hardware infrastructure [51]. These instances package the operating system, applications, and configurations needed for VMs.

5.2.1.2 VirtualBox

VirtualBox is an open-source virtualisation software developed by Oracle that allows users to create and run VMs on their computers. It acts as a Type 2 hypervisor that operates on top of an existing operating system to simulate additional operating systems within isolated environments [52]. VirtualBox was used to create a secure and isolated environment for executing ransomware samples and benign applications. This was essential for safely collecting HPC traces without risking damage to the host system or network.

5.2.1.3 PerfExtract

PerfExtract is a specialised tool designed to extract HPC traces from running programs, providing detailed insights into their microarchitectural behaviour. HPCs are registers built into modern processors that capture low-level performance metrics such as cache misses, branch mispredictions, CPU cycles, and instructions retired. PerfExtract leverages these counters to collect time-series data, enabling the analysis of program execution patterns for applications like ransomware detection. The tool generates output in the form of CSV files containing time-series data for each performance counter, which can then be used for further analysis.

5.2.2 Dataset Overview

The dataset is structured to facilitate deep-learning-based classification tasks, where the goal is to distinguish ransomware from benign applications based on HPC metrics. The dataset comprises two categories, namely, malicious samples and benign samples. There are 414 CSV files containing HPC traces for malicious executables and 288 CSV files containing HPC traces for benign applications. Each CSV file contains 23 columns, representing the different HPC counters used and 60 rows corresponding to 60-time steps, sampled at intervals of 0.5 seconds over a total duration of 30 seconds. This structure ensures that each file provides a comprehensive snapshot of program behaviour over time capturing both short-term and long-term fluctuations in HPC metrics.

The dataset includes 23 distinct HPC counters that monitor various aspects of program execution. The counters used are summarised in the table below with their descriptions [53]:

HPC Counter	Description
% Privileged Time	Percentage of non-idle processors spent executing code in privileged mode.
Handle Count	The number of handles currently opened by a process.
IO Read Operations/sec	The rate of reads per second for files, network and I/O devices.
IO Data Operations/sec	The rate of reads and writes per second for files, network and I/O devices.
IO Write Operations/sec	The rate of writes per second for files, network and I/O devices.
IO Other Operations/sec	The rate of other I/O operations for files, network and I/O devices.
IO Read Bytes/sec	The rate of bytes read per second for files, network and I/O devices.
IO Write Bytes/sec	The rate of bytes issued to I/O operations per second for files, network and I/O devices.
IO Data Bytes/sec	The rate of bytes read and wrote I/O operations per second for

	files, network and I/O devices.
IO Other Bytes/sec	The rate of bytes used in control operations per second for files, network and I/O devices.
Page Faults/sec	The rate of page faults per second handled by the processor. It includes both types of page faults: disk page faults and memory page faults.
Page File Bytes Peak	The maximum amount of virtual memory, in bytes, reserved by the target process to be used for paging files.
Page File Bytes	The current amount of virtual memory, in bytes, reserved by the target process to be used for paging files.
Pool Paged Bytes	The number of bytes from the area of system memory that can be written to the disk.
Pool Non-paged Bytes	The number of bytes from the area of system memory that cannot be written to the disk.
Private Bytes	The size in bytes of the memory allocated that cannot be shared with other processes.
Priority Base	The current priority base for the target process.
Thread Count	The number of threads that are running in the target process.
Virtual Bytes Peak	The maximum size in bytes for the virtual address space used by the target process.
Virtual Bytes	The size in bytes for the virtual address space used by the target process.
Working Set Peak	The maximum size in bytes for the working memory set of the target process.
Working Set	The size in bytes of the working memory set for the target process.
Working Set - Private	Subset of working set reflecting the un-shared amount of memory.

Table 6: Summary of HPC counters used

The counters shown in Table 6, collectively capture critical aspects of system behaviour during program execution, providing a rich feature set for ransomware detection.

5.2.3 Data Pre-Processing

Data pre-processing is a critical step in preparing the dataset for effective training and optimisation of deep learning models. Due to the diverse range and scale of values collected from HPC counters, careful pre-processing techniques are required to ensure that neutral network models can efficiently learn meaningful patterns from the data.

5.2.3.1 Feature Scaling of HPC Dataset

HPC counter values inherently exhibit a wide range of numeral values due to the nature of the low-level hardware events they measure. These large variations in feature scales can negatively impact the training process of deep learning models, causing slower convergence and reduced model performance. To address this issue, feature scaling techniques were employed to normalise and standardise HPC values, thereby enabling efficient optimisation and improved model performance. Two common scaling techniques known as min-max normalisation and standardisation were applied in this project.

The min-max normalisation method rescales each feature within a specified range, typically between 0 and 1. It ensures that all features contribute equally during model training by eliminating biases introduced by varying scales. The min-max normalisation formula is defined below where X represents the original HPC feature value, X_{min} represents the minimum value of that feature, X_{max} represents the maximum value of that feature, and X' represents the normalised value within [0, 1]:

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Min-max normalisation is particularly beneficial for HPC data since it preserves relative differences between data points while standardising their scale for efficient learning.

The standardisation method transforms each feature such that it has a mean of zero and a standard deviation of 1. This method ensures that all features have comparable statistical probabilities, which facilitates faster convergence during neural network training. The formula is

defined below, where Xrepresents the original HPC feature value and X' represents the standardised value:

$$X' = \frac{X - X_{mean}}{X_{standard\ deviation}}$$

5.2.3.2 Dataset Conversion and Labelling

The raw dataset was provided as CSV files with 414 files representing malicious samples and 288 files representing benign samples. To prepare this data for deep learning models, it was converted into NumPy arrays with a shape of (n_samples, 60, 23) using the NumPy library in Python, as shown below in Figure 6 where:

- n_samples represents the total number of samples (e.g., 414 malicious and 288 benign samples).
- 60 represents the number of time steps per sample.
- 23 represents the number of HPC counters per time step.

This three-dimensional structure allows deep learning models to process temporal patterns across multiple HPC metrics simultaneously. Additionally, each sample was labelled as either 1 (malware) or 0 (benign). Labels were generated based on the directory from which each CSV file was loaded (malware or safe). The following Python snippet demonstrates how the dataset was converted into NumPy arrays and labelled as malware or safe:

```python
# Function to load a random selection of files and create data and labels
def load_dataset(dataset_path, sub_path, num_samples=288):
    # Initialize empty arrays for data and labels
    x = np.array([]).reshape(0, 60, 23)
    y = np.array([]).reshape(0, 1)

    # Get all available files in the given sub-path
    files = os.listdir(os.path.join(dataset_path, sub_path))
    selected_files = random.sample(files, min(num_samples, len(files)))

    for file in selected_files:
        # Read the CSV file
        df = pd.read_csv(os.path.join(dataset_path, sub_path, file)).to_numpy()

        # Reshape and append
        x = np.append(x, df.reshape(1, 60, 23), axis=0)

        # Assign labels: 1 for malware, 0 for safe
        if sub_path == "malware":
            y = np.append(y, [[1]], axis=0)
        else:
            y = np.append(y, [[0]], axis=0)

    return x, y
```

Figure 6: Dataset Conversion into NumPy Arrays & Labelling

This process ensured that each sample was labelled correctly based on its origin while maintaining compatibility with deep learning frameworks like TensorFlow.

5.2.3.3 Implementation of Scaling Techniques

After converting the data into NumPy arrays and assigning labels, scaling techniques were applied to normalise or standardise the HPC values. Figure 7 below demonstrates how scaling is implemented using scikit-learn:

```python
from sklearn.preprocessing import MinMaxScaler, StandardScaler
# Function to normalize data using Min-Max or Z-Score normalization
def data_normalization(data, standard=True):
    if standard:
        scaler = StandardScaler()
    else:
        scaler = MinMaxScaler()

    # Flatten data for normalization, then reshape back
    original_shape = data.shape
    data = scaler.fit_transform(data.reshape(original_shape[0] * 60, 23)).reshape(original_shape)
    return data
```

Figure 7: Code Implementation of Scaling Techniques

The function defined in Figure 7 supports both min-max normalisation and Z-score standardisation by specifying the standard parameter.

5.2.4 Dataset Imbalance and Resolution

The dataset used for ransomware detection exhibits an imbalance between the number of malicious and benign samples. Specifically, there are 414 malicious samples compared to 288 benign samples, leading to an unequal distribution of classes. This imbalance can negatively impact the performance of deep learning models as they may be biased towards the majority class of malicious samples during training. Consequently, the model may fail to generalise effectively and exhibit poor performance in detecting benign samples, resulting in higher false negative rates.

The disadvantages brought about by this imbalance are as follows:

1. **Model Bias:** The model tends to favour the majority class, leading to biased predictions. This could result in higher false negative rates where software is incorrectly classified as malicious.
2. **Reduced Generalisation:** The model may not learn adequate representations of the minority class features, leading to poor performance.
3. **Poor Performance Metrics:** Metrics such as precision, recall, and F1-score for the minority class may be adversely affected. A model could achieve high accuracy simply by classifying all samples as the majority class.
4. **Training Instability:** Imbalanced data can cause unstable gradients during training, potentially resulting in longer convergence time or suboptimal local minima.

To mitigate the issues associated with dataset imbalance, a balanced dataset approach was implemented. This involved randomly selecting 288 malware samples to match the number of benign samples, thereby creating a perfectly balanced dataset with 576 total samples. This ensured that both classes contributed equally during training, improving model generalisations and reducing bias. The following strategy was implemented in the load_dataset and load_data functions in the following code:

```python
# Function to load a random selection of files and create data and labels
def load_dataset(dataset_path, sub_path, num_samples=288):
    # Initialize empty arrays for data and labels
    x = np.array([]).reshape(0, 60, 23)
    y = np.array([]).reshape(0, 1)

    # Get all available files in the given sub-path
    files = os.listdir(os.path.join(dataset_path, sub_path))
    selected_files = random.sample(files, min(num_samples, len(files)))

    for file in selected_files:
        # Read the CSV file
        df = pd.read_csv(os.path.join(dataset_path, sub_path, file)).to_numpy()

        # Reshape and append
        x = np.append(x, df.reshape(1, 60, 23), axis=0)

        # Assign Labels: 1 for malware, 0 for safe
        if sub_path == "malware":
            y = np.append(y, [[1]], axis=0)
        else:
            y = np.append(y, [[0]], axis=0)

    return x, y

# Function to load, normalize, and split data into train/test/validation sets
def load_data(dataset_path, ratio, norm=False, num_samples=288):

    # Load only num_smaples random safe and num_smaples random malware datasets
    x_safe, y_safe = load_dataset(dataset_path, "safe", num_samples)
    x_malware, y_malware = load_dataset(dataset_path, "malware", num_samples)
```

Figure 8: Code Implementation of Dataset Imbalance Resolution

From the implementation in Figure 8, the num_samples parameter is set to 288 by default, ensuring that an equal number of samples is selected from each class. The random.sample() function selects a random subset of files when the number of available files exceeds the specified num_samples. As such, a balanced dataset was achieved comprising 50% malware and 50% benign samples.

5.2.5 Dataset Partitioning for Deep Learning

A crucial aspect of developing robust deep-learning models is the appropriate partitioning of the dataset. The dataset partitioning strategy implemented employs a three-way split, dividing the data into training, testing, and validation sets. This approach enables:

1. **Model Training:** The training set (70% of data) is used to optimise mode parameters through backpropagation.

2. **Model Evaluation:** The testing set (20% of data) provides an unbiased evaluation of model performance on unseen data.
3. **Hyperparameter Tuning:** The validation set (10% of data) allows for the optimisation of hyperparameters without contaminating the test set.

The three-way dataset partitioning is implemented in the load_data function in Figure 9 below:

```python
# Function to load, normalize, and split data into train/test/validation sets
def load_data(dataset_path, ratio, norm=False, num_samples=288):

    # Load only num_smaples random safe and num_smaples random malware datasets
    x_safe, y_safe = load_dataset(dataset_path, "safe", num_samples)
    x_malware, y_malware = load_dataset(dataset_path, "malware", num_samples)

    # Normalize data for both classes
    x_safe = data_normalization(x_safe, standard=norm)
    x_malware = data_normalization(x_malware, standard=norm)

    # Combine data and labels
    x = np.concatenate((x_safe, x_malware), axis=0)
    y = np.concatenate((y_safe, y_malware), axis=0)

    # Shuffle the combined data
    x, y = shuffle(x, y)

    # Split into train, test, and validation sets
    X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=ratio[1], random_state=8)
    X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=ratio[2]/(1. - ratio[1]), random_state=9)

    return X_train, X_test, X_val, y_train, y_test, y_val

def normalize_balanced_dataset(dataset_path=r"../Dataset", save_directory=r"../LLM", norm=False, num_samples=288):
    # Load and normalize the dataset
    ratios = [0.7, 0.2, 0.1]  # Ratios for train/test/validation
    X_train, X_test, X_val, y_train, y_test, y_val = load_data(dataset_path, ratios, norm, num_samples)
```

Figure 9: Code Implementation of Dataset Partitioning

Illustrated in Figure 9, the data is first split into preliminary training and test sets, 80% and 20% respectively. Then, the preliminary training set is further divided to create the final training set (70%) and validation set (10%). Fixed random seeds, “random_state=8” and “random_state=9”, were used to enable deterministic splitting to ensure reproducibility of the splits. The normalize_balanced_dataset function then saves these partitioned datasets for subsequent model development, illustrated in Figure 10 below:

```python
def normalize_balanced_dataset(dataset_path=r"../Dataset", save_directory=r"../LLM", norm=False, num_samples=288):
    # Load and normalize the dataset
    ratios = [0.7, 0.2, 0.1]  # Ratios for train/test/validation
    X_train, X_test, X_val, y_train, y_test, y_val = load_data(dataset_path, ratios, norm, num_samples)

    # Save datasets for later use
    np.save(os.path.join(save_directory, "train.npy"), X_train)
    np.save(os.path.join(save_directory, "test.npy"), X_test)
    np.save(os.path.join(save_directory, "val.npy"), X_val)
    np.save(os.path.join(save_directory, "train_labels.npy"), y_train)
    np.save(os.path.join(save_directory, "test_labels.npy"), y_test)
    np.save(os.path.join(save_directory, "val_labels.npy"), y_val)
```

Figure 10: Saving of Partitioned Dataset

5.3 Deep Learning Model Architectures

A total of 7 deep-learning models were developed for ransomware detection using HPC counters. Out of the 7 models, 3 were optimised using NAS.

5.3.1 Convolutional Neural Network (CNN)

The first deep learning model implemented for ransomware detection was a CNN, due to its exceptional capability in extracting spatial and temporal patterns from sequential data. The model implements a 1D convolutional approach, specifically tailored to process time series data where each sample consists of 23 feature channels recorded over 60 time steps.

The CNN model comprises two primary components, as depicted in Figure 11 below. The first component includes convolutional layers that extract specific features from the input HPC data using learnable filters. The second component consists of fully connected layers (dense layers) that combine these extracted features for binary classification, distinguishing between ransomware (1) and benign files (0).

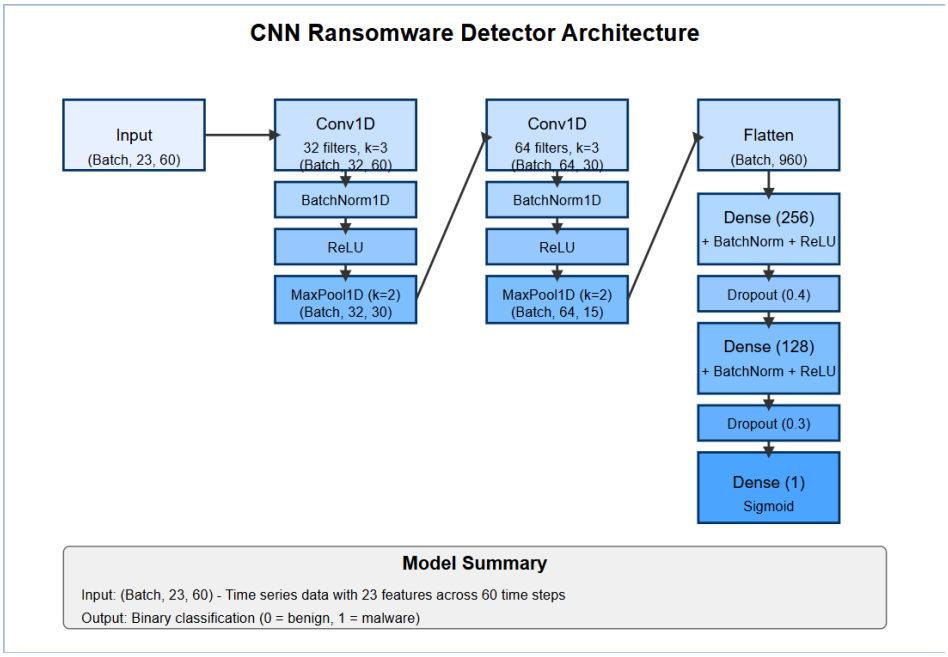

Figure 11: Graph Representation of CNN Model Architecture

The architecture was designed to balance complexity and efficiency, incorporating modern deep learning techniques such as batch normalisation, dropout regularisation, and ReLU activations to

enhance performance. The final output layer uses a sigmoid activation function to produce probabilities for binary classification.

The model was trained using the Adam optimiser with an initial learning rate of 0.001 and weight decay of 1e-5 for additional regularisation. A learning rate scheduler was implemented to dynamically adjust the learning rate during training. The scheduler reduces the learning rate by a factor of 0.5 when the validation loss fails to improve for seven consecutive epochs, enhancing convergence stability. Early stopping with a patience of 15 epochs was employed to prevent overfitting and unnecessary computations. This mechanism saves the model with the lowest validation loss and stops training when no improvement is observed for 15 consecutive epochs.

5.3.1.1 Input Layer

The input training data discussed in sections 5.3.2 and 5.6 first undergoes a crucial transformation before feeding into the network, illustrated below in Figure 12:

```
# Reshape for Conv1D
X_train = torch.tensor(X_train, dtype=torch.float32).permute(0, 2, 1)
X_val = torch.tensor(X_val, dtype=torch.float32).permute(0, 2, 1)
```

Figure 12: Data Transformation for CNN Input

This permutation operation rearranges the dimensions to match PyTorch's Conv1D layer expectation, which requires input in the form (batch_size, channel, sequence_length). In this context, the 23 HPC features serve as channels, while the 60-time steps represent the sequence length over which convolutional operations are performed.

5.3.1.2 Convolutional Layers and Feature Extraction

```
self.conv1 = nn.Conv1d(in_channels=23, out_channels=32, kernel_size=3, padding=1)
self.conv2 = nn.Conv1d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
self.pool = nn.MaxPool1d(kernel_size=2)
self.batch_norm1 = nn.BatchNorm1d(32)
self.batch_norm2 = nn.BatchNorm1d(64)
```

Figure 13: Code Implementation of CNN Convolutional Layers

The model employs a hierarchical feature extraction approach illustrated in Figure 13 above, consisting of two convolutional blocks. Each block contains a Conv1D layer followed by batch normalisation, ReLU activation, and max pooling.

The first block transforms the 23 input channels into 32 feature maps using 3x1 kernels. Padding is applied to maintain the temporal dimensions, resulting in an output shape of (batch_size, 32, 60). After ReLU activation, max pooling with a kernel size of 2 reduces the sequence length to 30, yielding an output of shape (batch_size, 32, 30). The second block further processes the 32 feature maps from the first block, extracting 64 higher-level feature maps. Similar to the first block, 3x1 kernels with padding are applied. After activation and pooling, the output shape becomes (batch_size, 64, 15).

5.3.1.3 Flattening and Fully Connected Layers

Following the convolutional feature extraction, the model flattens the feature maps and processes them through a series of fully connected layers, illustrated below in Figure 14:

```
self.flatten = nn.Flatten()
self.fc1 = nn.Linear(self.fc_input_size, 256)
self.batch_norm_fc1 = nn.BatchNorm1d(256)
self.fc2 = nn.Linear(256, 128)
self.batch_norm_fc2 = nn.BatchNorm1d(128)
self.fc3 = nn.Linear(128, 1)
```

Figure 14: Code Implementation of CNN Fully Connected Layers

The flattening operation converts the three-dimensional output from the convolutional layers (batch_size, 64, 15) into a two-dimensional tensor of shape (batch_size, 960). The fully connected layers progressively reduce the dimensionality from 960 to a single output unit. Each fully connected layer, except the output layer, is followed by batch normalisations and ReLU activation.

5.3.1.4 Regularisation Techniques

To mitigate overfitting and enhance generalisation, the model incorporates dropout regularisation between fully connected layers, shown below in Figure 15:

```
# Droupout layers with different rates
self.dropout1 = nn.Dropout(0.4)
self.dropout2 = nn.Dropout(0.3)
```

Figure 15: Dropout Implementation for CNN

These dropout layers randomly deactivate 40% and 30% of the neurons in their respective layers during training, effectively preventing co-adaptation of features and encouraging the network to learn more robust representations.

5.3.1.5 Output Layer and Binary Classification

The final layer of the network consists of a single neuron with sigmoid activation, shown below in Figure 16:

```
x = torch.sigmoid(self.fc3(x))
return x.squeeze() # Squeeze to match the shape of the labels
```

Figure 16: Sigmoid Activation of Final Layer

The sigmoid activation function maps the network output to a probability value between 0 and 1, representing the likelihood of a sample being malware. The squeeze operation ensures that the output shape matches the labels during training, facilitating the computation of binary cross-entropy loss.

5.3.2 Hybrid CNN-RNN Neural Network

The second deep learning model implemented was a hybrid CNN-RNN as this approach combines the strengths of both CNNs and RNNs where CNNs excel at extracting spatial features and RNNs are particularly effective at capturing temporal dependencies in sequential data.

The CNN-RNN architecture consists of the following three main components, illustrated below in Figure 17:

1. **CNN Feature Extraction Module:** This module employs 2D convolution to process the initial input and extract special features from HPC data.
2. **RNN Sequence Learning Module:** Utilises multiple BigRU layers to capture temporal dependencies and sequential patterns.
3. **Classification Head:** Processes the features extracted by the RNN module to make the final binary classification decision.

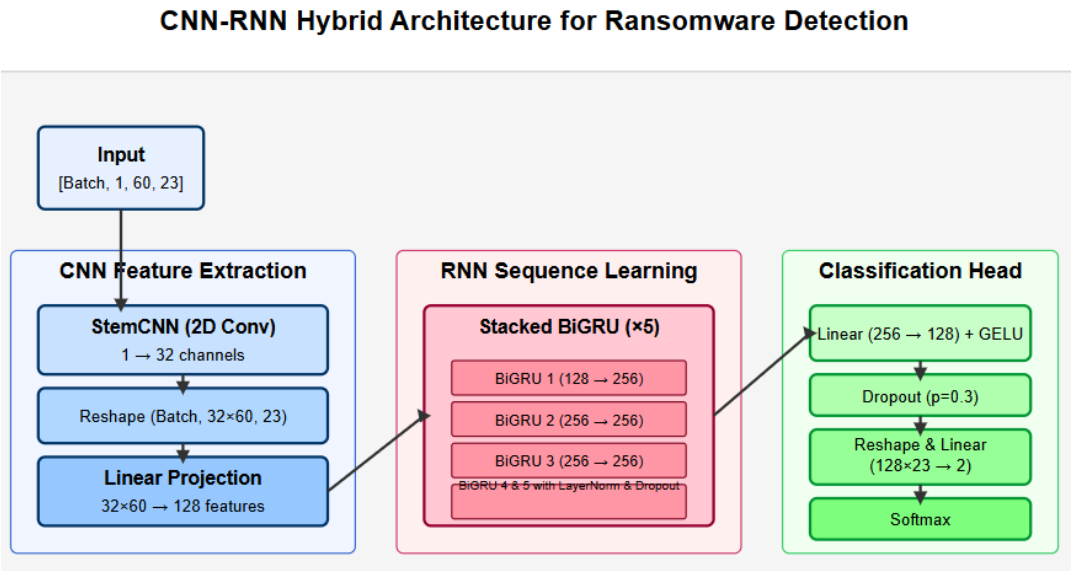

Figure 17: Graph Representation of Hybrid CNN-RNN Model Architecture

The model was trained using the AdamW optimiser with a learning rate of 1e-4 and a cosine annealing learning rate scheduler. The cosine annealing scheduler gradually reduces the learning rate following a cosine curve, facilitating better convergence by initially allowing larger parameter updates for faster learning and later enabling more precise fine-tuning with smaller

updates. Moreover, model checkpointing, loss computations using the standard cross-entropy loss function and early stopping based on accuracy were implemented.

5.3.2.1 Input Layer

For processing through the 2D convolutional layer, the input data was reshaped to include a channel dimension, illustrated below in Figure 18:

```
# input_sequences shape: [batch, 60, 23]; add channel dim → [batch, 1, 60, 23]
input_sequences = input_sequences.unsqueeze(1).to(device)
```

Figure 18: Data Transformation for CNN-RNN Input

This transformation creates a single-channel 2D "image", where rows represent time steps and columns represent different hardware counters.

5.3.2.2 CNN Feature Extraction Module

The initial 2D convolution, with a kernel size of 3x3, stride of 1, and padding of 1, transforms the single-channel input into a feature map with feat_dim channels (32 by default). The convolution operation preserves the spatial dimensions while expanding the channel dimension, allowing the network to detect various patterns across both the temporal and feature dimensions. Following the convolution, the data undergoes a critical reshaping operation, illustrated below in Figure 19:

```
x = self.stemcnn(x)
sizes = x.size()
# print(x.size())
x = x.view(sizes[0], sizes[1]*sizes[2], sizes[3])
```

Figure 19: Data Reshaping After Convolutional Layer

This reshaping flattens the channel and temporal dimensions, transforming the tensor from the shape of (batch_size, 32, 60, 23) to (batch_size, 32x60, 23). The resulting structure represents each time step's features expanded into the higher-dimensional space created by the convolutional filters.

The final component of the CNN module is a linear projection that maps the expanded features to the dimension expected by the RNN module, illustrated below in Figure 20:

```
self.fc = nn.Linear(feat_dim * input_height, rnn_dim)
x = x.transpose(1, 2) # x = [batch, length, channel (features)]
x = self.fc(x)
```

Figure 20: Linear Projection for Feature Mapping

This projection reduces the dimensionality from 32x60 to rnn_dim (128 by default), preparing the data for sequential processing by the RNN.

5.3.2.3 RNN Sequential Learning Module

The RNN module consists of multiple bidirectional Gated Recurrent Unit (BiGRU) layers to capture temporal dependencies. The architecture utilizes 5 stacked BigRU layers and each layer is implemented as illustrated in Figure 21:

```
class BiGRU(nn.Module):
    def __init__(self, rnn_dim, hidden_size, dropout, batch_first):
        super(BiGRU, self).__init__()

        self.bigru = nn.GRU(input_size=rnn_dim, hidden_size=hidden_size, \
            num_layers=1, batch_first=batch_first, bidirectional=True)
        self.layer_norm = nn.LayerNorm(rnn_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        x = self.layer_norm(x)
        x = F.gelu(x)
        x, _ = self.bigru(x)
        x = self.dropout(x)
        return x
```

Figure 21: Code Implementation of RNN BiGRU Layers

Each BiGRU layer incorporates a GELU activation function, layer normalisation, bidirectional GRU and dropout. The bidirectional nature of the GRU layers is particularly valuable as it ensures that the model can detect patterns that might manifest early or late in the training phase or have complex interdependencies across different points in time.

5.3.2.4 Classification Head

The classification head processes the high-level features extracted by the RNN module to make the final prediction as illustrated in Figure 22:

```
self.classifier = nn.Sequential(
    nn.Linear(rnn_dim*2, rnn_dim),
    nn.GELU(),
    nn.Dropout(dropout),
    # nn.Linear(rnn_dim, n_class)
)
self.fc2 = nn.Linear(rnn_dim*input_len, n_class)
```

Figure 22: Code Implementation of Classification Head

The linear layer maps the combined features to the two output classes, ransomware or benign. During training and inference, the output is processed through a Log-Softmax function to produce class probabilities.

5.3.3 Long Short-Term Memory (LSTM) Neural Network

The third deep learning model implemented was an LSTM to capture the temporal dependencies present in HPC feature data. LSTM networks are particularly well-suited for this task as they excel at processing sequential data and can capture long-range dependencies through their gating mechanism. The overall LSTM architecture is illustrated in Figure 23 below:

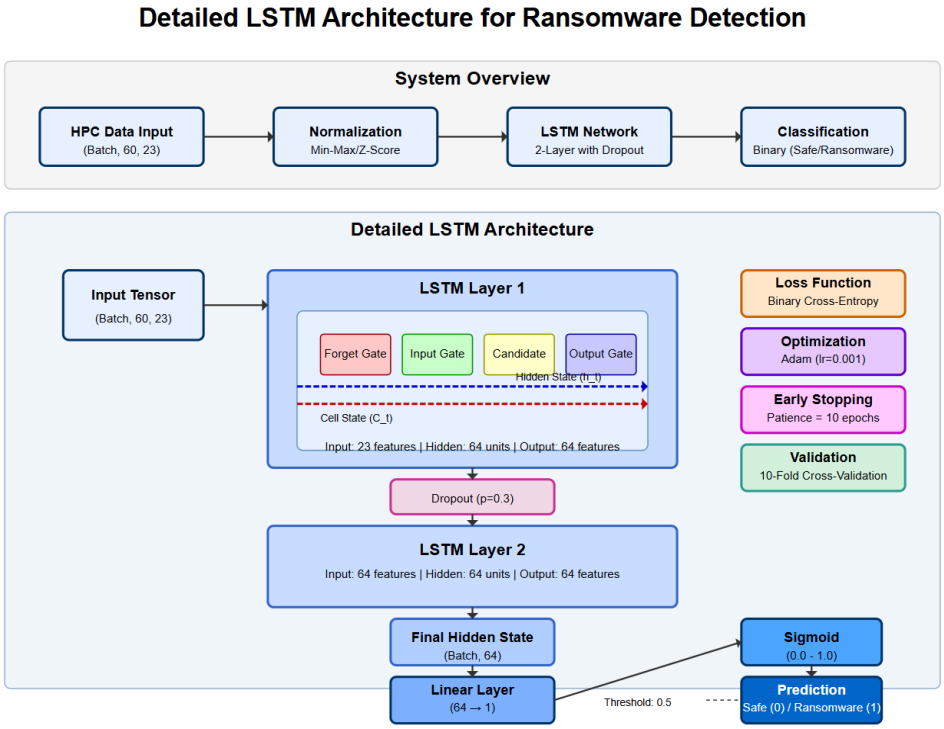

Figure 23: Graph Representation of LSTM Model Architecture

The model was designed with the following architecture:

1. **Stacked LSTM Layers**: Two LSTM layers with a hidden size of 64 units each process the input sequence. The first layer takes the 23 HPC features as input at each time step, while the second layer processes the 64-dimensional hidden states from the first layer.
2. **Dropout Regularization:** A dropout rate of 0.3 is applied between the LSTM layers to prevent overfitting, randomly deactivating 30% of the connections during training.
3. **Classification Head:** The final hidden state from the last LSTM layer is passed through a single fully connected layer that maps from the 64-dimensional hidden space to a single output node.

4. **Sigmoid Activation:** The output is passed through a sigmoid function to produce a probability value between 0 and 1, where values closer to 1 indicate a higher likelihood of the sample being malware.
5. **K-Fold Cross-Validation**: The model was validated using 10-fold cross-validation to ensure robust performance assessment.
6. **Binary Cross-Entropy Loss**: The binary nature of the classification task (malware vs. benign) was addressed with BCE loss.
7. **Adam Optimizer**: Training utilized the Adam optimizer with a learning rate of 0.001.
8. **Early Stopping**: To prevent overfitting, training was halted when validation loss failed to improve for 10 consecutive epochs.
9. **Batch Processing**: Data was processed in batches of 64 samples to improve training efficiency.

5.3.3.1 Data Flow and Processing

```
class LSTMRansomwareDetector(nn.Module):
    def __init__(self, input_size=23, hidden_size=64, num_layers=2, dropout=0.3):
        super(LSTMRansomwareDetector, self).__init__()
        self.lstm = nn.LSTM(input_size=input_size, hidden_size=hidden_size,
                            num_layers=num_layers, batch_first=True, dropout=dropout)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        # x: (batch, seq_len, input_size)
        lstm_out, (hn, cn) = self.lstm(x)
        # Use the hidden state from the last LSTM layer (last time step)
        last_hidden = hn[-1]
        out = self.fc(last_hidden)
        return torch.sigmoid(out).squeeze()
```

Figure 24: Code Implementation of LSTM Model

From Figure 24, the flow of data through the LSTM model can be analysed. The input time series data (batch_size, 60, 23) is fed into the first LSTM layer. Each LSTM layer processes the sequence and produces both output sequences and final hidden states. Only the final hidden state from the last LSTM layer is extracted for classification. This hidden state is passed through a fully connected layer and sigmoid activation to produce the final classification. The forward pass extracts the final hidden state (hn[-1]) rather than using the complete output sequence, focusing the classification decision on the cumulative information gathered across the entire time series.

5.3.4 Transformer

The fourth deep learning model implemented for ransomware detection was a Transformer, which offers distinct advantages over traditional recurrent models through its parallel processing capabilities and attention mechanisms. This parallel processing approach allows the model to capture complex dependencies between HPC metrics at different time points without being constrained by the sequential information flow of recurrent architectures. The overall architectural overview of the model can be seen below in Figure 25:

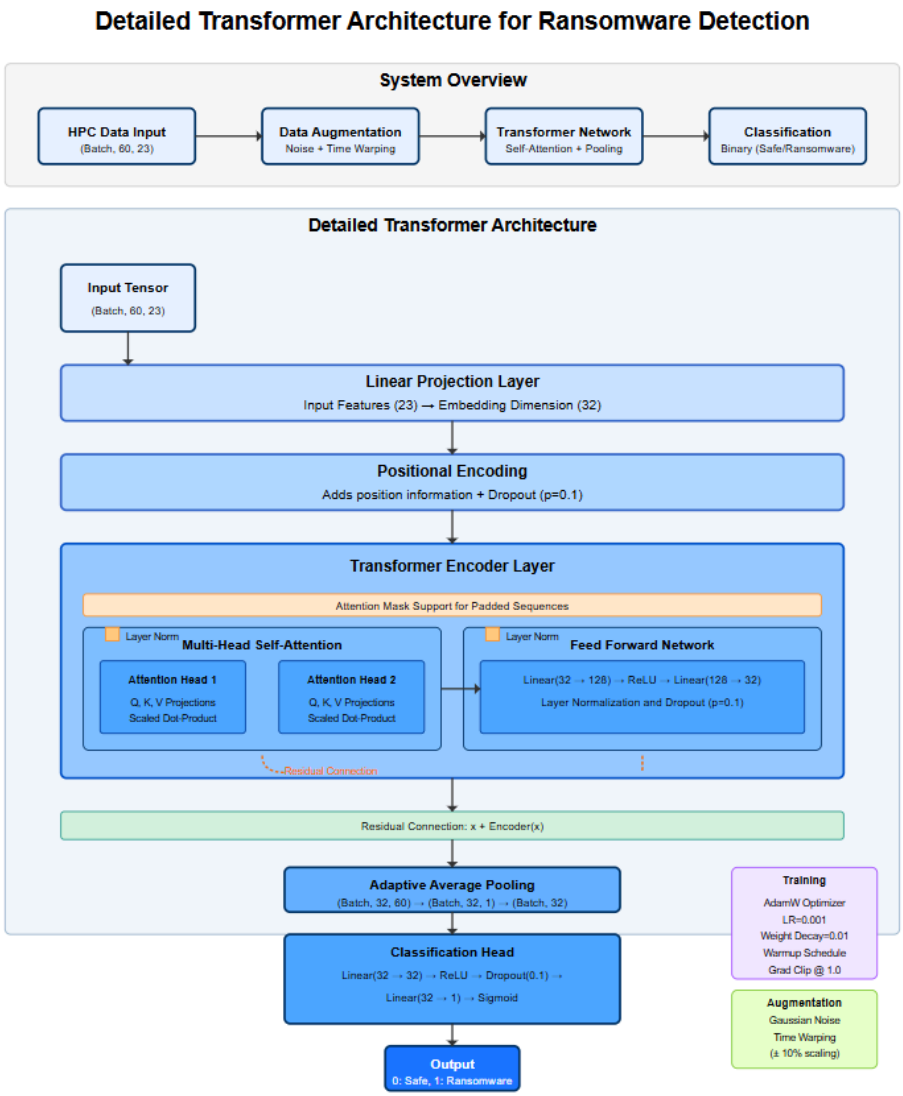

Figure 25: Graph Representation of Transformer Model Architecture

Based on Figure 25, the transformer-based ransomware detector follows a sequential processing pipeline that transforms HPC data into binary classification decisions. The architecture consists of the following four main components:

1. **Input Processing:** Performs linear transformations of raw HPC features into a higher-dimensional embedding space.
2. **Positional Encoding:** Addition of position information to preserve sequential context.
3. **Transformer Encoder:** Self-attention mechanism for capturing relationships between time points
4. **Classification Head:** Consists of adaptive pooling and fully connected layers for binary classification.

The model is trained using the AdamW optimiser that applied L2 regularisation with a weight decay of 0.01 to prevent overfitting. Early stopping is used to prevent overfitting with a patience value of 15 epochs and training terminates when validation loss fails to improve.

5.3.4.1 Input Projection Layer

Before processing the input data through the self-attention mechanism, the input features are projected into a higher-dimensional embedding space using a linear transformation shown below in Figure 26:

```
self.input_proj = nn.Linear(num_features, d_model)
```

Figure 26: Linear Transformation of Input Data

This projection transforms the original 23 HPC features into a 32-dimensional embedding space (d_model=32), creating improved feature representations that can better capture the behaviour of ransomware.

5.3.4.2 Positional Encoding

The transformer architecture processes all time steps in parallel meaning it lacks inherent sequential ordering information. To reintroduce this critical temporal context, positional encodings are added to the input embeddings, shown below in Figure 27:

```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=500, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-np.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + self.pe[:, :x.size(1)]
        return self.dropout(x)
```

Figure 27: Code Implementation of Positional Encoding for Transformer Model

The positional encoding employs a sinusoidal function that generates unique position=dependent patterns.

5.3.4.3 Transformer Encoder and Self-Attention Mechanism

The core component of the architecture is the transformer encoder, which processes the positionally encoded embeddings through self-attention mechanisms, shown below in Figure 28:

```python
encoder_layer = nn.TransformerEncoderLayer(
    d_model=d_model,
    nhead=nhead,
    dropout=dropout,
    batch_first=True
)
self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
```

Figure 28: Code Implementation of Transformer Encoder

The transformer's encoder uses a single encoder layer with two attention heads to process the sequence in parallel, each focusing on different aspects of the temporal relationships. A dropout rate of 0.1 prevents overfitting within the transformer's encoder components.

The self-attention mechanism enables each time step to attend to all other time steps in the sequence. This allows the model to identify correlations between HPC values at different points during the program execution. The attention mechanism computes the weights that determine the importance of other time steps relevant to the current one.

5.3.4.4 Classification Head

After processing through the transformer encoder, the resulting sequence representations are condensed into a fixed-size vector through adaptive average pooling. The pooling operation aggregates information across all time steps, creating a comprehensive representation of the entire sequence. The pooled representation is then processed through a series of fully connected layers for the final classification as illustrated in Figure 29 below:

```
self.pool = nn.AdaptiveAvgPool1d(1)
self.fc = nn.Sequential(
    nn.Linear(d_model, d_model),
    nn.ReLU(),
    nn.Dropout(dropout),
    nn.Linear(d_model, 1)
)
```

Figure 29: Code Implementation of Transformer Classification Head

The classification head uses a ReLU activation function with a dropout rate of 0.1. The final linear layer projects to a single output dimension for binary classification. A sigmoid activation function is used to convert the raw score to probability values between 0 and 1.

5.4 NAS Implementation for Neural Network Optimisation

In this section, NAS was implemented using the Optuna Python Library, a hyperparameter optimisation framework to discover the optimal neural network architectures for CNNs, LSTMs and Transformers. The NAS implementation follows the following systematic process:

1. **Search Space Definition:** Identifying the architectural components and hyperparameters to optimise
2. **Search Strategy:** Employing efficient algorithms to explore the search space.

3. **Evaluation Strategy:** Defining metrics to assess potential architectures.
4. **Model Training and Selection:** Training potential architectures and selecting the best-performing architecture.

The Optuna NAS algorithm for the three models utilised the same search which employed the Tree-structured Parzen Estimator (TPE) algorithm as its search strategy. TPE is a Bayesian optimisation technique that leverages information from previous trials to guide subsequent exploration, focusing computational resources on promising regions of the search space. Optuna uses this algorithm to build models that remember which architectural configurations worked well in the past. When choosing new candidate architectures, Optuna tries to favour configurations that have a higher chance of success based on past results. An example of the search process is defined in the run_neural_architecture_search function, illustrated below in Figure 30:

```python
# Main NAS procedure
def run_neural_architecture_search(n_trials=50):
    study_name = "cnn_ransomware_nas"
    storage_name = f"sqlite:///{study_name}.db"

    study = optuna.create_study(
        study_name=study_name,
        storage=storage_name,
        direction="minimize",
        load_if_exists=True
    )

    logger.info(f"Starting Neural Architecture Search with {n_trials} trials")
    start_time = time.time()

    study.optimize(objective, n_trials=n_trials)

    end_time = time.time()

    logger.info(f"NAS completed in {(end_time - start_time)/60:.2f} minutes")
```

Figure 30: Code Implementation of NAS Search Strategy

While the code implementation in Figure 30 is from the NAS optimisation of the CNN network, the other two networks make use of the same search strategy in their optimisations with a slightly different code implementation due to the difference in network architectures.

Additionally, the same evaluation strategy was used by Optuna for the optimisation of all three networks. The objective function used for the NAS implementation was validation loss with early stopping implemented to prevent overfitting, illustrated below in Figure 31:

```
# Train and validate model
best_val_loss, _, _ = train_and_validate(
    model, train_loader, val_loader, optimizer, criterion, scheduler, patience
)

return best_val_loss
```

Figure 31: Code Implementation of NAS Evaluation Strategy

This approach optimises the model's ability to generalise to unseen data, rather than simply fitting the training data. The validation loss provides a more reliable indicator of real-world performance than training loss or accuracy metrics alone.

5.4.1 NAS Implementation for CNN Optimisation

The first neural network architecture that was optimised using NAS was the CNN network discussed in Section 5.7.1.

5.4.1.1 CNN Search Space Definition

The search space defined for CNN included filter counts, kernel sizes, pooling sizes, dropout rates, and the number of convolutional and fully connected layers shown below in Figure 32:

```python
def objective(trial):
    # Architecture parameters
    num_conv_layers = trial.suggest_int("num_conv_layers", 1, 3)

    # Dynamically create lists based on number of conv layers
    filters = []
    kernel_sizes = []
    pool_sizes = []
    dropout_rates = []

    # Sample parameters in a way that prevents invalid architectures
    for i in range(num_conv_layers):
        filters.append(trial.suggest_int(f"filters_{i}", 16, 128, step=16))
        kernel_sizes.append(trial.suggest_int(f"kernel_size_{i}", 3, 7, step=2))

        # Restrict pool sizes to prevent sequence length from becoming too small
        if i == 0:
            # For first layer, we can use pool sizes up to 3
            pool_sizes.append(trial.suggest_int(f"pool_size_{i}", 2, 3))
        else:
            # For later layers, be more conservative with pool size
            pool_sizes.append(2)  # Fixed pool size of 2 for later layers

        dropout_rates.append(trial.suggest_float(f"conv_dropout_{i}", 0.1, 0.5))

    # FC layers parameters
    num_fc_layers = trial.suggest_int("num_fc_layers", 1, 3)

    fc_layers = []
    fc_dropout_rates = []

    for i in range(num_fc_layers):
        fc_layers.append(trial.suggest_int(f"fc_size_{i}", 64, 512, step=64))
        fc_dropout_rates.append(trial.suggest_float(f"fc_dropout_{i}", 0.1, 0.5))
```

Figure 32: Search Space Definition for NAS CNN

Additionally, the search space included training hyperparameters such as batch sizes, learning rates, weight decays, early stopping patience and batch normalisation as shown below in Figure 33:

```python
# Training parameters
batch_size = trial.suggest_categorical("batch_size", [16, 32, 64])
learning_rate = trial.suggest_float("learning_rate", 1e-4, 1e-2, log=True)
weight_decay = trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True)
patience = trial.suggest_int("patience", 10, 20)
use_batch_norm = trial.suggest_categorical("use_batch_norm", [True, False])
```

Figure 33: Hyperparameter Search Space for NAS CNN

A key aspect of the implementation involved the validation of the candidate architectures before training, as shown below in Figure 34. This prevents resource expenditure on invalid architectures that would fail during execution.

```python
# Function to check if architecture is valid before creating model
def is_valid_architecture(num_conv_layers, pool_sizes):
    sequence_length = 60
    for i in range(num_conv_layers):
        sequence_length = sequence_length // pool_sizes[i]
        if sequence_length <= 0:
            return False
    return True
```

Figure 34: Validation of Candidate Architectures

The function defined in Figure 34 ensures that sequential pooling operations do not reduce the sequence length to zero which would cause model failures.

5.4.1.2 Dynamic CNN Model Implementation

The CNN architecture was implemented with configurable components to adapt to different search parameters by creating a DynamicCNNRansomwareDetector class, as shown below in Figure 35:

```python
class DynamicCNNRansomwareDetector(nn.Module):
    def __init__(self,
                 num_conv_layers=2,
                 filters=[32, 64],
                 kernel_sizes=[3, 3],
                 pool_sizes=[2, 2],
                 dropout_rates=[0.3, 0.3],
                 fc_layers=[256, 128],
                 fc_dropout_rates=[0.4, 0.3],
                 use_batch_norm=True):
        super(DynamicCNNRansomwareDetector, self).__init__()

        self.use_batch_norm = use_batch_norm
        self.conv_layers = nn.ModuleList()
        self.pool_layers = nn.ModuleList()
        self.dropout_layers = nn.ModuleList()
        self.batch_norm_layers = nn.ModuleList()
```

Figure 35: Code Implementation of DynamicCNNRansomwareDetector Class

The flexible class in Figure 35 dynamically constructed convolutional, pooling, and normalisation layers based on the architectural parameters. The training strategy incorporated

several techniques for stable and efficient learning such as the Adam optimiser, adaptive learning rate scheduling, early stopping with patience based on validation loss, and model checkpointing to preserve the best-performing configuration.

5.4.1.3 Optimal CNN Architecture Discovery

After systematic exploration of the search space through 50 trials, the NAS process identified an optimal architecture with the following configurations:

```
Best Model Architecture:

Number of Conv Layers: 1
Conv Layer 1: Filters=16, Kernel=3, Pool=2, Dropout=0.21029145048004166

Number of FC Layers: 2
FC Layer 1: Units=192, Dropout=0.1829248250226745
FC Layer 2: Units=128, Dropout=0.1439940386895310 3

Batch Size: 16
Learning Rate: 0.00395413719088903
Weight Decay: 3.935179884303537e-05
Early Stopping Patience: 19
Batch Normalization: True
Activation Function: Relu/Sigmoid
```

Figure 36: Optimal CNN Architecture Discovered By NAS

The architecture discovered in Figure 36, prioritises simplicity over depth, with just one convolutional layer and two fully connected layers.

5.4.2 NAS Implementation for LSTM Optimisation

The second neural network architecture that was optimised using NAS was the LSTM network discussed in Section 5.7.3.

5.4.2.1 LSTM Search Space Definition

The search space defined for the LSTM included dropout rates, bidirectional configurations, number of LSTM layers, LSTM hidden sizes and number and size of fully connected layers, shown below in Figure 37:

```python
# LSTM architecture parameters
num_lstm_layers = trial.suggest_int("num_lstm_layers", 1, 3)
lstm_hidden_sizes = []

# Generate a list of decreasing hidden sizes
first_hidden_size = trial.suggest_categorical("first_hidden_size", [32, 64, 128, 256])
lstm_hidden_sizes.append(first_hidden_size)

for i in range(1, num_lstm_layers):
    # Each subsequent layer has fewer units
    prev_size = lstm_hidden_sizes[i-1]
    size_options = [s for s in [16, 32, 64, 128] if s <= prev_size]
    if not size_options:  # Fallback if no valid options
        size_options = [16]
    hidden_size = trial.suggest_categorical(f"lstm_hidden_size_{i}", size_options)
    lstm_hidden_sizes.append(hidden_size)

lstm_dropout = trial.suggest_float("lstm_dropout", 0.1, 0.5)
bidirectional = trial.suggest_categorical("bidirectional", [True, False])

# Fully connected layers
num_fc_layers = trial.suggest_int("num_fc_layers", 0, 2)
fc_layers = []

for i in range(num_fc_layers):
    fc_size = trial.suggest_categorical(f"fc_size_{i}", [16, 32, 64, 128])
    fc_layers.append(fc_size)

fc_dropout = trial.suggest_float("fc_dropout", 0.1, 0.5)
```

Figure 37: Search Space Definition for NAS LSTM

Additionally, the search space included training hyperparameters such as batch sizes, learning rates, weight decays, early stopping patience, batch normalisation, and activation functions as shown below in Figure 38:

```python
# Other hyperparameters
batch_size = trial.suggest_categorical("batch_size", [16, 32, 64, 128])
learning_rate = trial.suggest_float("learning_rate", 1e-4, 1e-2, log=True)
weight_decay = trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True)
use_batch_norm = trial.suggest_categorical("use_batch_norm", [True, False])
activation = trial.suggest_categorical("activation", ["relu", "leaky_relu", "elu", "tanh"])
patience = trial.suggest_int("patience", 10, 20)
```

Figure 38: Hyperparameter Search Space for NAS LSTM

5.4.2.2 Dynamic LSTM Implementation

To support the diverse architectures defined in the search space, a flexible DynamicLSTMRansomwareDetector class, capable of configuring itself based on the sampled hyperparameters was implemented, illustrated below in Figure 39:

```python
class DynamicLSTMRansomwareDetector(nn.Module):
    def __init__(self,
                 input_size=23,
                 hidden_sizes=[64, 32],
                 num_layers=2,
                 dropout=0.3,
                 bidirectional=False,
                 fc_layers=[64, 32],
                 fc_dropout=0.3,
                 use_batch_norm=True,
                 activation='relu'):

        super(DynamicLSTMRansomwareDetector, self).__init__()

        self.use_batch_norm = use_batch_norm
        self.bidirectional = bidirectional
        self.num_directions = 2 if bidirectional else 1
        self.hidden_sizes = hidden_sizes
        self.num_layers = num_layers
        self.activation_name = activation

        if activation == 'relu':
            self.activation = nn.ReLU()
        elif activation == 'leaky_relu':
            self.activation = nn.LeakyReLU(0.1)
        elif activation == 'elu':
            self.activation = nn.ELU()
        else:
            self.activation = nn.Tanh()
```

Figure 39: Code Implementation of DynamicLSTMRansomwareDetector Class

This flexible class in Figure 39 has the following design aspects:

1. **Dynamic LSTM Stack**: Multiple LSTM layers are created based on the specified hidden sizes.
2. **Batch Normalisation Support:** Optional batch normalisation layers follow each LSTM layer to stabilise training,
3. **Configurable Fully Connected Layers:** The model dynamically creates fully connected layers based on the architecture parameters.

The training strategy incorporated several techniques for stable and efficient learning such as adaptive learning rate scheduling, early stopping based on validation loss, and model checkpointing to preserve the best-performing configuration.

5.4.2.3 Optimal LSTM Architecture Discovery

After systematic exploration of the search space through 100 trials, the NAS process identified an optimal LSTM architecture with the following configurations:

```
Number of LSTM Layers: 1
LSTM Hidden Sizes: [32]
Bidirectional: True
LSTM Dropout: 0.4863511687193945

Number of FC Layers: 1
FC Layer Sizes: [32]
FC Dropout: 0.15097275011801622

Batch Size: 16
Learning Rate: 0.006944785741985918
Weight Decay: 0.00010051077433485776
Use Batch Normalization: True
Activation Function: tanh
Early Stopping Patience: 17
Total Parameters: 16,769
```

Figure 40: Optimal LSTM Architecture Discovered By NAS

The architecture discovered in Figure 40, favours a single LSTM layer with bidirectional processing suggesting that additional recurrent layers do not provide significant benefits for ransomware detection.

5.4.3 NAS Implementation for Transformer Optimisation

The third neural network architecture that was optimised using NAS was the Transformer network discussed in Section 5.7.4.

5.4.3.1 Search Space Definition

The search space defined for the Transformer included dimensions of the model (d_model), number of attention heads in multi-head attention (nhead), number of encoder layers (num_layers), dimensions of the feedforward network (dim_feedforward), dropout rates, positional encoding, residual connections, pooling types and number and size of fully connected layers, shown below in Figure 41:

```
# Transformer architecture parameters
d_model = trial.suggest_categorical("d_model", [16, 32, 64, 128, 256, 512])
nhead_options = [2, 4, 8]  # Ensure nhead divides d_model evenly
valid_nheads = [h for h in nhead_options if d_model % h == 0]
nhead = trial.suggest_categorical("nhead", valid_nheads)
num_layers = trial.suggest_int("num_layers", 1, 6)
dim_feedforward = trial.suggest_categorical("dim_feedforward", [64, 128, 256, 512])

# Regularization parameters
dropout = trial.suggest_float("dropout", 0.1, 0.5)
use_positional_encoding = trial.suggest_categorical("use_positional_encoding", [True, False])
use_residual = trial.suggest_categorical("use_residual", [True, False])

# Pooling strategy
pooling_type = trial.suggest_categorical("pooling_type", ["mean", "max", "cls", "adaptive_avg", "adaptive_max"])

# FC layers configuration
num_fc_layers = trial.suggest_int("num_fc_layers", 0, 3)
```

Figure 41: Search Space Definition for NAS Transformer

Additionally, the search space included training hyperparameters such as batch sizes, learning rates, weight decays, activations functions, augmentations, gradient clipping, patience values and schedulers, as shown below in Figure 42:

```
# Learning hyperparameters
batch_size = trial.suggest_categorical("batch_size", [16, 32, 64, 128])
learning_rate = trial.suggest_float("learning_rate", 1e-5, 1e-3, log=True)
weight_decay = trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True)
activation = trial.suggest_categorical("activation", ["relu", "gelu"])

# Training options
use_augmentation = trial.suggest_categorical("use_augmentation", [True, False])
clip_grad_norm_val = trial.suggest_categorical("clip_grad_norm", [0.0, 0.5, 1.0, 5.0])
patience = trial.suggest_int("patience", 10, 20)
use_scheduler = trial.suggest_categorical("use_scheduler", [True, False])
```

Figure 42: Hyperparameter Search Space for NAS Transformer

5.4.3.2 Dynamic Transformer Implementation

To support the diverse architectures defined in the search space, a flexible DynamicTransformerRansomwareDetector class, capable of configuring itself based on the various architectural parameters was implemented, illustrated below in Figure 43:

```python
class DynamicTransformerRansomwareDetector(nn.Module):
    def __init__(self,
                 num_features=23,
                 seq_len=60,
                 d_model=64,
                 nhead=4,
                 num_layers=2,
                 dim_feedforward=128,
                 dropout=0.1,
                 activation="relu",
                 layer_norm_eps=1e-5,
                 use_positional_encoding=True,
                 fc_sizes=[64, 32],
                 use_residual=True,
                 pooling_type="mean"):
        super().__init__()

        # Model architecture parameters
        self.num_features = num_features
        self.seq_len = seq_len
        self.d_model = d_model
        self.nhead = nhead
        self.num_layers = num_layers
        self.dim_feedforward = dim_feedforward
        self.dropout_rate = dropout
        self.use_positional_encoding = use_positional_encoding
        self.use_residual = use_residual
        self.pooling_type = pooling_type
```

Figure 43: Code Implementation of DynamicTransformerRansomwareDetector Class

This flexible class defined in Figure 43 has the following design aspects:

1. **Input Projection:** Performs linear transformations of the raw features to model dimensions.
2. **Optional Positional Encoding:** Addition of positional information which can be configured.
3. **Dynamic Transformer Encoder Layers:** Multiple encoder layers with configurable parameters.
4. **Multiple Pooling Options:** Choose from different strategies for sequence aggregation.

This flexible implementation enables efficient exploration of the diverse search space defined for the Transformer Architecture.

5.4.3.3 Optimal Transformer Architecture Discovery

After systematic exploration of the search space through 100 trials, the NAS process identified an optimal architecture with the following configurations:

```
Number of Transformer Layers: 3
Model Dimension (d_model): 32
Number of Attention Heads (nhead): 2
Feedforward Dimension: 512
Activation Function: gelu
Dropout Rate: 0.4258251381649406
Use Positional Encoding: False
Use Residual Connections: True
Pooling Strategy: mean

FC Layer Sizes: [32, 16]

Training Parameters:
Batch Size: 16
Learning Rate: 0.0009847304889551945
Weight Decay: 0.00033469493881814444
Use Data Augmentation: True
Gradient Clipping Value: 0.5
Early Stopping Patience: 20
Use Learning Rate Scheduler: False
Total Parameters: 115,361
```

Figure 44: Optimal Transformer Architecture Discovered By NAS

The architecture discovered in Figure 44, favours a 32-dimensional model space that consists of a three-layer encoder stack without positional encoding. The model made use of a 512-dimensional feedforward network with a GELU activation function after which, mean pooling was performed.

5.5 Evaluation of Deep Learning Models

Model evaluation represents a critical phase in the development of effective ransomware detection models, providing a quantitative measure of how well the trained models perform on previously unseen data. This section describes the evaluation methodology employed for assessing the performance of deep learning models designed in this project, focusing on metrics that reveal their efficacy in distinguishing between ransomware and benign applications based on HPC data.

5.5.1 Evaluation Methodology

To evaluate the deep learning models, a systematic approach was implemented that leveraged the dataset partitioning strategy outlined in Section 5.2.5. After each model was trained on the

training dataset, using 70% of the dataset, their performance was assessed using the independent test dataset which consisted of 20% of the dataset that was kept completely separate from the training process. This separation is crucial to obtain unbiased estimates of model performance, as it ensures that the models are evaluated on data they have never encountered during training thereby simulating real-world deployment scenarios.

The test dataset maintains the same balanced distribution of ransomware and benign samples as the training set, enabling a fair assessment of the model's capabilities across both classes. During evaluation, each model processed the test samples and generated predictions that were then compared against the true labels to calculate various performance metrics. This approach provides a robust estimate of how well the models would perform when deployed in a real-world environment for ransomware detection.

5.5.2 Confusion Matrix Components

The foundation of model evaluation for this binary classification task is the confusion matrix, which categorises predictions into the following four main components:

1. **True Positive (TP):** TP represents the number of ransomware samples that have been correctly identified as ransomware. This metric indicates the model's ability to detect actual threats.

2. **True Negative (TN):** TN represents the number of benign samples that have been correctly identified as benign. This metric indicates the model's ability to avoid raising false alarms.

3. **False Positive (FP):** FP represents the number of benign samples incorrectly identified as ransomware. These false alarms can cause legitimate applications to be blocked or unnecessary remediation action, potentially disrupting user operations and reducing confidence in the detection system.

4. **False Negative (FN):** FN represents the number of ransomware samples incorrectly identified as benign. These missed detections are particularly concerning in the cybersecurity context, as they allow malicious software to operate without being detected, potentially leading to data encryption, financial loss, and other detrimental consequences.

These four components serve as the basis for calculating more complex performance metrics that can provide greater insights into different aspects of model performance evaluation.

5.5.3 Performance Metrics

The following complementary performance metrics were employed to thoroughly evaluate the models' performance, each highlighting different aspects of their efficacy in ransomware detection:

1. **Accuracy:**

Accuracy measures the proportion of correct predictions among all the predictions made, providing a general indication of model performance. It is calculated as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

While accuracy offers an intuitive measure of overall performance, it can be misleading if an imbalanced dataset is used, where one class significantly outnumbers the other. However, since the dataset used in this project was balanced as described in Section 5.2.4, accuracy serves as a reliable metric for comparing model performance.

2. **Precision:**

Precision quantifies the proportion of positive predictions that are actually correct, indicating the model's ability to avoid false alarms, it is calculated as follows:

$$Precision = \frac{TP}{TP + FP}$$

High precision indicates that when the model identifies a sample as ransomware, it will likely be correct. This metric is vital in contexts where FP have significant costs, such as wrongly quarantining legitimate applications that users need for their operations.

3. **Recall:**

Recall, also referred to as sensitivity, measures the proportion of actual positives that are correctly identified, indicating the model's ability to detect all ransomware samples. It is calculated as follows:

$$Recall = \frac{TP}{TP + FN}$$

High recall indicates that the model can successfully detect most or all ransomware applications. This metric is particularly critical in cybersecurity applications where missing a threat (FN) can have severe consequences, such as allowing ransomware to encrypt files or compromise system security.

4. **F1 Score**

F1 score combines precision and recall into a single metric, providing a balance between the two. It is particularly useful when seeking a balance between detecting all threats and avoiding false alarms. The F1 score is calculated as a harmonic mean of precision and recall as follows:

$$F1\ Score = \frac{2 \times (Precision \times Recall)}{Precision + Recall}$$

This harmonic mean ensures that the F1 score gives equal weights to both precision and recall, making it a robust metric for evaluating models where both FPs and FNs can have significant implications. A high F1 score indicates that the model achieves both high precision and recall, effectively balancing the trade-off between these two metrics.

5. **Matthews Correlation Coefficient (MCC)**

The MCC is a balanced measure of the quality of binary classification, even when the binary classes are of different sizes. It takes into account TP, TN, FP and FN, providing a more reliable summary of the confusion matrix with an output between -1 and +1. It is calculated as follows:

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

A coefficient of +1 represents a perfect prediction while a coefficient of -1 represents an inverse prediction. MCC accounts for all four confusion matrix elements in a single metric, ensuring that models are not overfitting to any particular outcome. This is crucial for real-world security applications where both FPs and FNs carry significant consequences.

The evaluation metrics described above were applied to all seven deep learning models developed in this project, enabling a comprehensive comparison of their performance in ransomware detection.

6. Results and Discussion

This evaluation of the deep learning modules implemented in this project reveals their effectiveness in detecting ransomware using HPCs. This section presents a comprehensive analysis of the performance of each model, followed by a comparative evaluation to determine the most effective approach for ransomware detection.

6.1 Model Performance Analysis

This section analyses the performance of all seven models used in this project.

6.1.1 Convolutional Neural Network Results

The CNN model achieved an accuracy of 97.41% on the test dataset, demonstrating strong performance in distinguishing between ransomware and benign applications. As shown below in Figure 45, the confusion matrix indicates that the model correctly identified 54 ransomware samples (TP) and 59 benign samples (TN), with 3 ransomware samples incorrectly classified as safe (FN) and no benign samples misclassified as ransomware (FP).

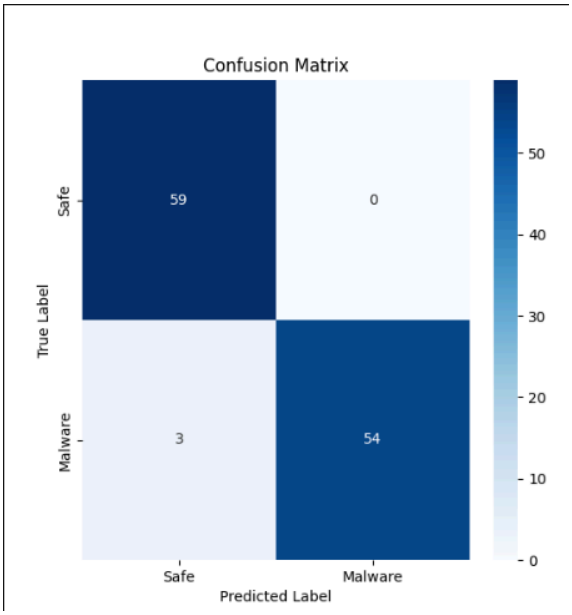

Figure 45: CNN Confusion Matrix

The classification report of the CNN is as follows:

```
🔹 Classification Report:
              precision    recall  f1-score   support

         0.0       0.95      1.00      0.98        59
         1.0       1.00      0.95      0.97        57

    accuracy                           0.97       116
   macro avg       0.98      0.97      0.97       116
weighted avg       0.98      0.97      0.97       116
```

Figure 46: CNN Classification Report

From the classification report, the model demonstrated high precision (98%) and recall (97%), resulting in an F1 score of 97%. These metrics indicate that the CNN effectively balances the ability to detect ransomware samples while minimising false alarms. The CNN model achieved an MCC score of 0.950, indicating a strong correlation between predictions and actual labels.

6.1.2 Hybrid CNN-RNN Neural Network Results

The hybrid CNN-RNN model achieved an accuracy of 97.41% on the test dataset, demonstrating strong performance in distinguishing between ransomware and benign applications. As shown below in Figure 47, the confusion matrix indicates that the model correctly identified 64 ransomware samples (TP) and 49 benign samples (TN), with 1 ransomware sample incorrectly classified as benign (FN) and 2 benign samples misclassified as ransomware (FP).

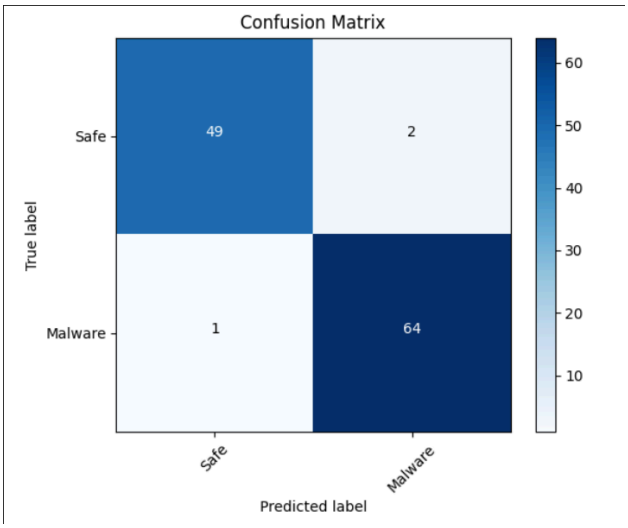

Figure 47: Hybrid CNN-RNN Confusion Matrix

The classification report of the hybrid CNN-RNN network is as follows:

```
Classification Report:
              precision    recall  f1-score   support

        Safe       0.98      0.96      0.97        51
     Malware       0.97      0.98      0.98        65

    accuracy                           0.97       116
   macro avg       0.97      0.97      0.97       116
weighted avg       0.97      0.97      0.97       116
```

Figure 48: Hybrid CNN-RNN Classification Report

From the classification report, the model exhibited balanced precision (97%) and recall (97%), resulting in an F1 score of 97%. The hybrid nature of this model, combining CNN's spatial feature extraction capabilities with RNN's ability to capture temporal dependencies, proved effective for analysing the sequential HPC data. Moreover, this model had a lower FN compared to the CNN model. Evaluation using the MCC metric resulted in a score of 0.948. While slightly lower than the CNN model's MCC, this score still demonstrates excellent performance.

6.1.3 LSTM Neural Network Results

The LSTM model achieved an impressive accuracy of 98.28% on the test dataset, outperforming the base CNN and hybrid CNN-RNN models. As shown below in Figure 49, the confusion matrix indicates that the model correctly identified 57 ransomware samples (TP) and 57 benign samples (TN), with no ransomware samples incorrectly classified as benign (FN) and 2 benign samples misclassified as ransomware (FP).

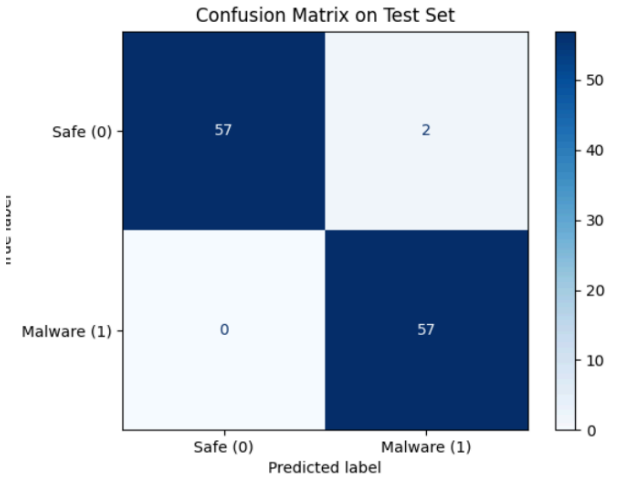

Figure 49: LSTM Confusion Matrix

The classification report of the LSTM network is as follows:

```
Classification Report:
              precision    recall  f1-score   support

        Safe     1.0000    0.9661    0.9828        59
     Malware     0.9661    1.0000    0.9828        57

    accuracy                         0.9828       116
   macro avg     0.9831    0.9831    0.9828       116
weighted avg     0.9833    0.9828    0.9828       116
```

Figure 50: LSTM Classification Report

From the classification report, the model demonstrated a precision of 96.61% and perfect recall (100%) for ransomware samples, resulting in an F1 score of 98.28%. These metrics indicate that the LSTM's ability to capture long-term dependencies in sequential data makes it particularly well-suited for detecting patterns in HPC traces associated with ransomware behaviour. The

LSTM model achieved an impressive MCC score of 0.966, surpassing both the CNN and hybrid CNN-RNN models.

6.1.4 Transformer Model Results

The Transformer model achieved an accuracy of 98.28% on the test dataset, matching the performance of the LSTM model. As shown below in Figure 51, the confusion matrix indicates that the model correctly classified 55 ransomware samples (TP) and 59 benign samples (TN), with 2 ransomware samples incorrectly classified as benign (FN) and no benign samples misclassified as ransomware (FP).

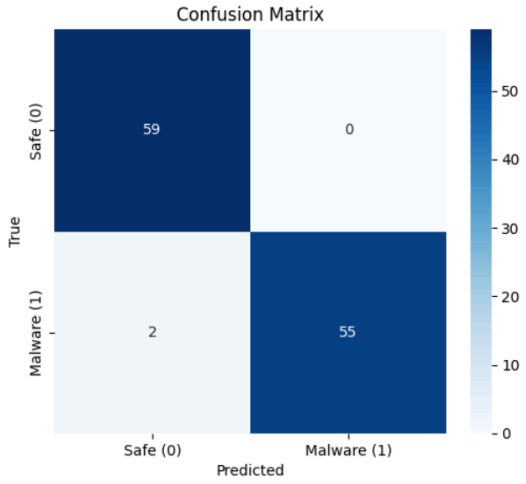

Figure 51: Transformer Confusion Matrix

The classification report of the Transformer model is as follows:

```
Evaluation Metrics:
Accuracy:  0.9828
Precision: 1.0000
Recall:    0.9649
F1-Score:  0.9821

Classification Report:
              precision    recall  f1-score   support

         0.0     0.9672    1.0000    0.9833        59
         1.0     1.0000    0.9649    0.9821        57

    accuracy                         0.9828       116
   macro avg     0.9836    0.9825    0.9827       116
weighted avg     0.9833    0.9828    0.9827       116
```

Figure 52: Transformer Classification Report

From the classification report, the model exhibited perfect precision (100%) for ransomware detection and a recall of 96.49%, resulting in an F1 score of 98.21%. The Transformer's self-attention mechanism proved effective in capturing relevant dependencies in the HPC data, demonstrating its suitability for ransomware detection tasks. With an MCC score of 0.966, the Transformer model demonstrated an excellent correlation between predicted and actual classes.

6.1.5 NAS-Optimised CNN Model

The NAS-optimised CNN model achieved an outstanding accuracy of 99.14% on the test dataset, representing a significant improvement over the manually designed CNN. As shown below in Figure 53, the confusion matrix reveals near-perfect classification, with 57 ransomware samples correctly identified (TP), 58 benign samples correctly identified (TN), no ransomware samples incorrectly classified as benign (FN), and 1 benign sample misclassified as ransomware (FP).

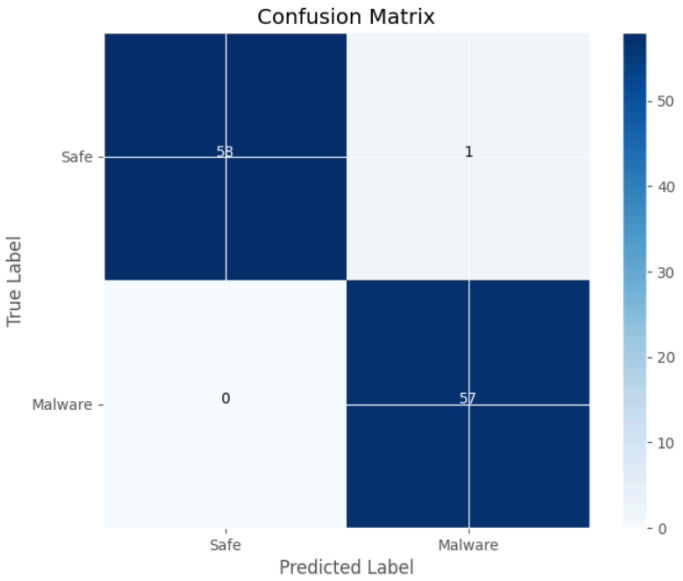

Figure 53: NAS-optimised CNN Confusion Matrix

The classification report of the NAS-optimised CNN model is as follows:

Classification Report:

	precision	recall	f1-score	support
0.0	1.00	0.98	0.99	59
1.0	0.98	1.00	0.99	57
accuracy			0.99	116
macro avg	0.99	0.99	0.99	116
weighted avg	0.99	0.99	0.99	116

Figure 54: NAS-optimised CNN Classification Report

From the classification report, the model demonstrated perfect precision (100%) and high recall (98%), resulting in an F1 score of 99%. The NAS process identified an optimal architecture consisting of just one convolutional layer and two fully connected layers, highlighting that simpler architectures can sometimes outperform more complex ones when properly optimised. The NAS-optimised CNN had an MCC score of 0.983, therefore it demonstrated superior performance compared to the manually designed CNN model.

6.1.6 NAS-Optimised LSTM Model

The NAS-optimised LSTM model achieved an accuracy of 99.14% on the test dataset, matching the performance of the NAS-optimised CNN. As shown below in Figure 55, the confusion matrix reveals almost perfect classification, with 56 ransomware samples correctly identified (TP), 59 benign samples correctly identified (TN), 1 ransomware sample incorrectly classified as benign (FN), and no benign samples misclassified as ransomware (FP).

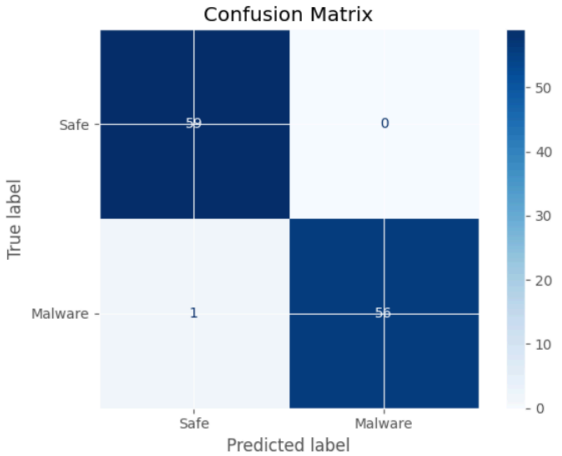

Figure 55: NAS-optimised LSTM Confusion Matrix

The classification report of the NAS-optimised LSTM model is as follows:

```
Test Metrics:
Test Loss: 0.0177
Test Accuracy: 0.9914
Test Precision: 1.0000
Test Recall: 0.9825
Test F1 Score: 0.9912

Confusion Matrix:
[[59  0]
 [ 1 56]]

Classification Report:
              precision    recall  f1-score   support

        Safe       0.98      1.00      0.99        59
     Malware       1.00      0.98      0.99        57

    accuracy                           0.99       116
   macro avg       0.99      0.99      0.99       116
weighted avg       0.99      0.99      0.99       116
```

Figure 56: NAS-optimised LSTM Classification Report

From the classification report, the model demonstrated perfect precision (100%) for ransomware detection and high recall (98.25%), resulting in an F1 score of 99.12%. The NAS process identified an optimal architecture featuring a single bidirectional LSTM layer with a hidden size of 32, followed by a single fully connected layer with 32 units. The NAS-optimised LSTM model achieved an equally impressive MCC score of 0.983, indicating near-perfect correlations between predictions and actual labels.

6.1.7 NAS-Optimised Transformer Model

The NAS-optimised Transformer model achieved an accuracy of 98.28% on the test dataset. As shown below in Figure 57, the confusion matrix revealed that the model correctly classified 55 ransomware samples (TP) and 59 benign samples (TN), with 2 ransomware samples incorrectly classified as benign (FN) and no benign samples misclassified as ransomware (FP).

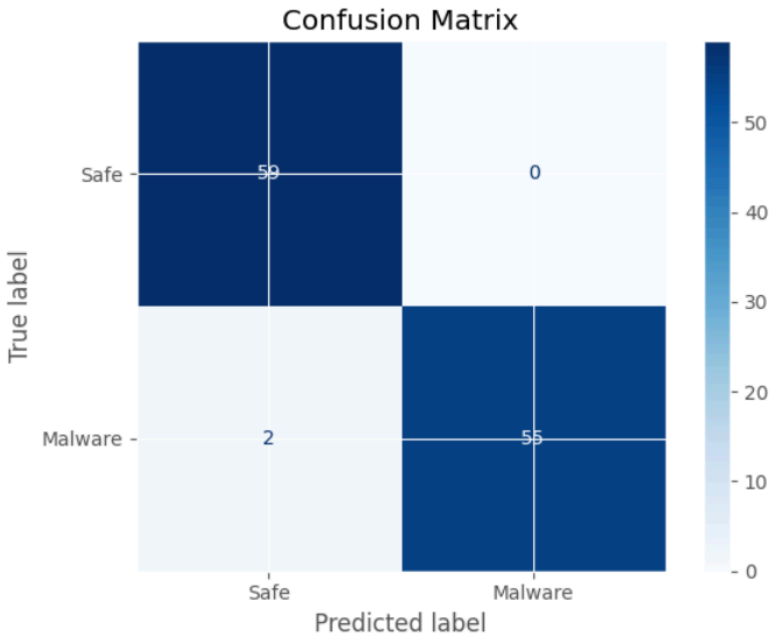

Figure 57: NAS-optimised Transformer Confusion Matrix

The classification report of the NAS-optimised Transformer model is as follows:

```
Test Metrics:
Test Loss: 0.8740
Test Accuracy: 0.9828
Test Precision: 1.0000
Test Recall: 0.9649
Test F1 Score: 0.9821

Classification Report:
              precision    recall  f1-score   support

        Safe       0.97      1.00      0.98        59
     Malware       1.00      0.96      0.98        57

    accuracy                           0.98       116
   macro avg       0.98      0.98      0.98       116
weighted avg       0.98      0.98      0.98       116
```

Figure 58: NAS-optimised Transformer Classification Report

From the classification report, the model demonstrated perfect precision (100%) for ransomware detection and a recall of 96.49%, resulting in an F1 score of 98.21%. The NAS process identified an optimal architecture featuring 3 transformer encoder layers, a model dimension of 32, and 2 attention heads, without positional encoding. With an MCC score of 0.966, the NAS-optimised Transformer model demonstrated a strong correlation between predicted and actual classes.

6.2 Comparative Analysis

A comprehensive comparison of all seven models is presented in Table 7 below, highlighting their respective performance metrics on the test dataset:

Model	Accuracy	Precision	Recall	F1 Score	Parameters	MCC
CNN	97.41%	0.98	0.97	0.97	288,449	0.950
Hybrid CNN-RNN	97.41%	0.9742	0.97	0.97	1,671,234	0.948
LSTM	98.28%	0.9661	1.00	0.98	56,129	0.966
Transformer	98.28%	1.00	0.9649	0.98	139,361	0.966
NAS-CNN	99.14%	0.99	0.99	0.99	118,977	0.983
NAS-LSTM	99.14%	1.00	0.9825	0.99	16,769	0.983
NAS-Transformer	98.28%	1.00	0.9649	0.98	115,361	0.966

Table 7: Summary Of Deep Learning Model Performance Metrics

The comparative analysis reveals several key findings:

1. **NAS-Optimized Models:** The models optimized using Neural Architecture Search consistently outperformed their manually designed counterparts, with the NAS-CNN and NAS-LSTM achieving the highest accuracy of 99.14%.

2. **Model Complexity vs. Performance:** Interestingly, the hybrid CNN-RNN model, despite having the highest parameter count (over 1.6 million parameters), did not achieve the best performance. In contrast, the NAS-LSTM model achieved superior performance with only 16,769 parameters, demonstrating that architectural optimization is more important than model complexity.

3. **Effectiveness of RNN-based Architectures:** Models that incorporated recurrent mechanisms (LSTM, hybrid CNN-RNN) generally performed well, highlighting the importance of capturing temporal dependencies in HPC data for ransomware detection.

4. **Precision-Recall Trade-off:** Several models achieved perfect precision (1.00), indicating they never misclassified benign applications as ransomware, which is a desirable characteristic for operational deployment to avoid disrupting legitimate software usage.

5. **Parameter Efficiency:** The NAS-LSTM model stands out for its exceptional parameter efficiency, achieving the highest accuracy with the fewest parameters. This has significant implications for deploying ransomware detection systems in resource-constrained environments.

6. **MCC Performance:** The MCC scores further validate the effectiveness of the NAS-optimised models, with the NAS-CNN and NAS-LSTM achieving the highest MCC (0.983) among all models. Interestingly, the Transformer and NAS-Transformer models achieved identical MCC scores of 0.966, suggesting that the baseline Transformer architecture was already well-optimised for this task.

Overall, the NAS-optimized models demonstrated superior performance with the NAS-LSTM achieving the best balance of accuracy, parameter efficiency, and computational requirements. This confirms the hypothesis that automated architecture optimization can significantly enhance the effectiveness of deep learning models for ransomware detection using HPCs.

6.3 Benchmarking

To assess the performance of the deep learning models developed in this project, their evaluation metrics were benchmarked against state-of-the-art (SOTA) ransomware detection frameworks reported in various academic literature. This benchmarking provides a comparative analysis of the proposed models' effectiveness relative to industry-leading approaches, highlighting their strengths and weaknesses.

The table below summarises the performance metrics of the proposed deep learning models alongside SOTA ransomware detection frameworks such as….

Model	Accuracy
NAS-CNN	99.14%
NAS-LSTM	99.14%
HiPeR [10]	98.68%
RanStop [10]	97%
DeepWare [10]	98.6%
RansomWall [54]	98.25%

Table 8: Comparison of Accuracy Metrics with SOTA Ransomware Detection Frameworks

Based on Table 8, the benchmarking results demonstrate that the NAS-optimised models developed in this project outperform current SOTA frameworks regarding detection accuracy. Both the NAS-CNN and NAS-LSTM models achieved an accuracy of 99.14%, representing an improvement of 0.46 percentage points over HiPeR, which was the next best-performing framework at 98.68%.

This benchmarking analysis confirms that the models developed in this project represent a meaningful advancement in ransomware detection performance. The improvements, while modest in percentage terms, are significant in the context of cybersecurity applications where

even small gains in detention accuracy can translate to substantial reductions in successful attacks.

6.4 Discussion of Results

The results of this study demonstrated the effectiveness of deep learning approaches for ransomware detection using HPCs. This section discusses several important observations and insights from the performance results:

1. **Effectiveness of HPC-Based Detection:**

The high accuracy achieved across all models, ranging from 96.41% to 99.14%, provides evidence that HPCs provide distinctive signatures that can reliably differentiate between ransomware and benign applications. This effectiveness of HPC-based detection is particularly noteworthy given the limitations of traditional sciatic and dynamic analysis discussed in Sections 2.4.1.2 and 2.4.2.2.

2. **Impact of NAS:**

The consistent improvement observed in NAS-optimised models compared to their manually designed counterparts validates the use of automated architecture optimisation for ransomware detection. The NAS algorithm effectively explored the search spaces of the different architectural networks to identify optimal designs which resulted in improved performance. This is particularly evident in the NAS-CNN model, which achieved greater performance with just one convolutional layer, contrary to the general belief that deeper networks generally perform better.

3. **Temporal vs Spatial Features in HPC Data:**

The strong performance of the recurrent models such as the LSTM model and hybrid CNN-RNN model, suggests that temporal dependencies in HPC data are significant indicators of ransomware behaviour. However, the comparable performance of the Transformer model, especially the NAS-optimised Transformer model without positional encoding, suggests that the absolute temporal ordering might be less critical than previously thought. This suggests that ransomware detection may rely more on the statistical distribution of HPC values rather than their exact sequences.

4. **Insights from MCC Metric:**

The MCC analysis provides additional confirmation of the models' effectiveness while offering nuanced insights about their performance characteristics. The consistently high MCC values across all models, ranging from 0.948 to 0.983, demonstrated that each architecture can effectively distinguish between ransomware and benign applications. The NAS-CNN and NAS-LSTM models achieved the highest MCC scores, which align with their excellent performance across the other metrics and further confirms their positions as the most effective architectures in this project.

6.5 Limitations

This section discusses the limitations of this project despite the promising results achieved by the deep learning models.

6.5.1 Dataset Limitations

The dataset used in this project has the following limitations:

1. **Sample Diversity:** The dataset contains a finite number of ransomware families and benign applications. In real-world scenarios, there exists a much broader spectrum of both ransomware variants and legitimate software, potentially limiting the generalisability of the findings in this project.

2. **Dataset Size:** With only 414 malicious samples and 288 benign samples, the dataset is relatively small compared to the vast diversity of samples available in real-world contexts. This limited dataset size could affect the models' ability to learn comprehensive and distinguishable patterns that represent the full spectrum of ransomware and benign behaviours.

3. **Controlled Environments:** The HPC traces were collected in a controlled virtual environment, which may not perfectly represent the real-world operating conditions where system load, background processes, and hardware variations could affect HPC measurements.

4. **Lack of Zero-Day Ransomware Samples:** The dataset does not contain zero-day ransomware samples that were unknown at the time of data collection. This presents a significant limitation, as the effectiveness of our models against previously unseen ransomware variants remains untested. Zero-day detection capability is particularly critical in the rapidly evolving ransomware landscape.

6.5.2 Implementation and Methodological Limitations

The following implementation and methodological limitations should be considered when analysing and interpreting the results of this project, which could give guidance to future works and implementations:

1. **Runtime Detection:** The models were evaluated offline on a pre-processed dataset rather than in real-time detection scenarios which would introduce additional challenges such as continuous monitoring overheads and timely intervention.

2. **System Integration:** The integration of these models into existing security frameworks and tools and their performance alongside other security measures was not explored.

3. **Binary Classification:** The models were designed for the binary classification of ransomware or benign applications, whereas real-world threat detection often requires multi-class classification to identify specific ransomware families or other malware types for better mitigation and detection approaches.

4. **NAS Search Space:** While NAS improved model performance, the defined search spaces were limited due to computational and resource constraints, potentially missing some optimal architectural configurations.

6.6 Future Work

Based on the findings and limitations of this project, there are several promising directions for future research works and studies:

1. **Enhanced Dataset:** Future work should focus on expanding the dataset to include a wider variety of ransomware families and benign applications, including those released after the current dataset was created.

2. **Advanced Model Architectures:** While this project explored several deep learning architectures, future works could investigate more sophisticated approaches such as ensemble methods. The method allows the combination of different models through voting or stacking which could potentially improve detection accuracy and robustness.

3. **Integration with Real-Time Detection Systems:** Integrating the deep learning models developed in this study into real-time detection systems could be a crucial next step. This could include optimising the models for deployment on resource-constrained devices, developing frameworks for continuous HPC monitoring and ransomware detection with minimal system overheads and integrating the detection models with automated response systems that can take immediate action when ransomware is detected such as process termination or system isolation.

7 Conclusion

This project has demonstrated the effectiveness of deep learning approaches for ransomware detection using HPCs. Through comprehensive experimentation with seven different model architectures, including both manually designed and NAS-optimised variants, we have shown that HPCs provide distinctive signatures that can reliably differentiate between ransomware and benign applications with high accuracy.

The NAS-optimised LSTM model emerged as the most efficient approach, achieving an accuracy of 99.14% with just 16,769 parameters. This highlights the potential for deploying lightweight, efficient ransomware detection systems that could operate with minimal computational overhead, addressing one of the key limitations of traditional dynamic analysis methods. Notably, our benchmarking against SOTA ransomware detection frameworks revealed that our NAS-optimised models outperform current leading approaches, including HiPeR (98.68%), DeepWare (98.6%), and RansomWall (98.25%). This improvement, while modest in percentage terms, is significant in the cybersecurity context where even small gains in detention accuracy can substantially reduce successful attacks.

The comparative analysis of different architectures revealed valuable insights about the nature of ransomware detection using HPCs. Recurrent models generally performed well, suggesting that temporal dependencies in HPC data are significant indicators of ransomware behaviour. However, the strong performance of the Transformer models without positional encoding indicates that the statistical distribution of HPC values might be more important than their exact sequence.

Despite the promising results, several limitations and challenges remain, including dataset diversity, real-time detecting implementation, and zero-day detection capabilities. These limitations point to fruitful directions for future research, including enhanced dataset exploration, advanced model architectures and real-time detection systems.

In conclusion, this project contributes to the growing body of research on hardware-based malware detection, demonstrating that microarchitectural events captured by HPCs, when

analysed through optimised deep learning models, provide an effective and efficient approach for ransomware detection. The improved performance of the models developed in this project, compared to existing frameworks, validates the combined use of HPCs and NAS for developing next-generation ransomware detention systems. This approach overcomes many limitations of traditional static and dynamic analysis methods, offering a promising avenue for enhancing cybersecurity defences against the growing threat of ransomware attacks.

8 References

[1] "Ransomware - Definition | Trend Micro (SG)," Trendmicro.com, 2024. https://www.trendmicro.com/vinfo/sg/security/definition/ransomware

[2] "Cybercrime To Cost The World $9.5 Trillion USD Annually In 2024," eSentire. https://www.esentire.com/web-native-pages/cybercrime-to-cost-the-world-9-5-trillion-usd-annually-in-2024

[3] "Incident Overview & Technical Details," Kaseya. https://helpdesk.kaseya.com/hc/en-gb/articles/4403584098961-Incident-Overview-Technical-Details

[4] M. Anghel and A. Racautanu, "A note on different types of ransomware attacks," Jun. 2019. Accessed: Oct. 21, 2024. [Online]. Available: https://eprint.iacr.org/2019/605

[5] Ö. Aslan and R. Samet, "A Comprehensive Review on Malware Detection Approaches," *IEEE Access*, vol. 8, no. 1, pp. 6249–6271, Jan. 2020, doi: https://doi.org/10.1109/ACCESS.2019.2963724.

[6] J. Podolanko, "Effective Crypto Ransomware Detection Using Hardware Performance Counters," MavMatrix, May 2019.

[7] G. Olani, C.-F. Wu, Y.-H. Chang, and W.-K. Shih, "DeepWare: Imaging Performance Counters with Deep Learning to Detect Ransomware," *IEEE Transactions on Computers*, pp. 1–14, 2022, doi: https://doi.org/10.1109/tc.2022.3173149.

[8] D. Sgandurra, L. Muñoz-González, R. Mohsen, and E. Lupu, "Automated Dynamic Analysis of Ransomware: Benefits, Limitations and use for Detection," Sep. 2016. Accessed: Mar. 22, 2025. [Online]. Available: https://arxiv.org/pdf/1609.03020

[9] M. Alam, S. Bhattacharya, D. Mukhopadhyay, and S. Bhattacharya, “Performance Counters to Rescue: A Machine Learning based safeguard against Micro-architectural Side-Channel-Attacks.” Accessed: Feb. 17, 2025. [Online]. Available: https://eprint.iacr.org/2017/564.pdf

[10] P. M. Anand, P. V. S. Charan, and S. K. Shukla, “HiPeR - Early Detection of a Ransomware Attack using Hardware Performance Counters,” *Digital Threats: Research and Practice*, vol. 4, no. 3, pp. 1–24, Sep. 2023, doi: https://doi.org/10.1145/3608484.

[11] Amazon, “Getting More Out of Amazon EC2 (30:59),” *Amazon Web Services, Inc.*, 2025. https://aws.amazon.com/pm/ec2/?gclid=CjwKCAiAkc28BhB0EiwAM001Tf8Iyz5roz6l07Ce5Dgp2At3TE-_9EJUycvUIJTODid0AZTH1CzLMhoC9qMQAvD_BwE&trk=3fc1271f-8d0f-43b5-b177-4fba4b680f8b&sc_channel=ps&ef_id=CjwKCAiAkc28BhB0EiwAM001Tf8Iyz5roz6l07Ce5Dgp2At3TE-_9EJUycvUIJTODid0AZTH1CzLMhoC9qMQAvD_BwE:G:s&s_kwcid=AL (accessed Jan. 24, 2025).

[12] Oracle, “Oracle VM VirtualBox,” *VirtualBox*, 2023. https://www.virtualbox.org/

[13] S. Wickramasinghe, “Ransomware Attacks Today: How They Work, Types, Examples & Prevention,” *Splunk-Blogs*, Aug. 22, 2024. https://www.splunk.com/en_us/blog/learn/ransomware-attack-types.html (accessed Jan. 25, 2025).

[14] C. Kidd, “Ransomware Families & RaaS Groups,” *Splunk*, Feb. 22, 2023. https://www.splunk.com/en_us/blog/learn/ransomware-families.html (accessed Jan. 30, 2025).

[15] M. Kosinski, “Ransomware,” *Ibm.com*, Jun. 04, 2024. https://www.ibm.com/think/topics/ransomware (accessed Jan. 30, 2025).

[16] RecordedFuture, “Most Popular Ransomware Groups to Watch (Updated 2024),” *Recordedfuture.com*, 2024. https://www.recordedfuture.com/threat-intelligence-101/cyber-threats/ransomware-groups (accessed Jan. 30, 2025).

[17] S. Yusirwan, Y. Prayudi, and I. Riadi, “Implementation of Malware Analysis using Static and Dynamic Analysis Method General Terms,” *International Journal of Computer Applications*, vol. 117, no. 6, pp. 1–5, 2015, Accessed: Feb. 15, 2025. [Online]. Available: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=ae14f0d365a74977efc23c8eb40270db8e880c1c

[18] Q. Kang and Y. Gu, “A Survey on Ransomware Threats: Contrasting Static and Dynamic Analysis Methods,” pp. 1–14, Nov. 2023, doi: https://doi.org/10.20944/preprints202311.0798.v1.

[19] A. Anand, “Extract strings,” *Securityinbits*, Apr. 29, 2018. https://www.securityinbits.com/malware-analysis/extract-strings/ (accessed Feb. 16, 2025).

[20] S. Ninja, “Static malware analysis | Infosec,” *www.infosecinstitute.com*, Apr. 29, 2015. https://www.infosecinstitute.com/resources/malware-analysis/malware-analysis-basics-static-analysis/ (accessed Feb. 16, 2025).

[21] D. Kostadinov, “Unlocking Cybersecurity: Exploring Reverse-Engineering & Malware Analysis Tools | Infosec,” *Infosecinstitute.com*, Feb. 04, 2020. https://www.infosecinstitute.com/resources/malware-analysis/reverse-engineering-and-malware-analysis-tools/

[22] S. Gujar and S. Patil, “A Machine Learning-Based PE Header Analysis for Malware Detection,” *International journal of innovative science and research technology*, vol. 9, no. 3, pp. 1671–1676, Mar. 2024, doi: https://doi.org/10.38124/ijisrt/ijisrt24mar615.

[23] R. Sihwail, K. Omar, and K. A. Zainol Ariffin, “A Survey on Malware Analysis Techniques: Static, Dynamic, Hybrid and Memory Analysis,” *International Journal on Advanced Science, Engineering and Information Technology*, vol. 8, no. 4–2, pp. 1–10, Sep. 2018, doi: https://doi.org/10.18517/ijaseit.8.4-2.6827.

[24] “syscalls(2) - Linux manual page,” *man7.org*. https://man7.org/linux/man-pages/man2/syscalls.2.html (accessed Mar. 04, 2025).

[25] N. Owoh, J. Adejoh, S. Hosseinzadeh, M. Ashawa, J. Osamor, and A. Qureshi, “Malware Detection Based on API Call Sequence Analysis: A Gated Recurrent Unit–Generative Adversarial Network Model Approach,” *Future Internet*, vol. 16, no. 10, p. 369, Oct. 2024, doi: https://doi.org/10.3390/fi16100369.

[26] A. Afianian, S. Niksefat, B. Sadeghiyan, and D. Baptiste, “Malware Dynamic Analysis Evasion Techniques,” *ACM Computing Surveys*, vol. 52, no. 6, pp. 1–28, Nov. 2019, doi: https://doi.org/10.1145/3365001.

[27] L. Su, H. Cheng, L. Li, C. Zhang, Y. Wang, and J. Zhao, “A Novel Approach of Ransomware Detection with Dynamic Obfuscation Signature Analysis ,” 2024, doi: https://doi.org/10.21203/rs.3.rs-5375812/v1.

[28] F. De Gaspari, D. Hitaj, Giulio Pagnotta, L. De Carli, and L. Mancini, “Evading behavioral classifiers: a comprehensive analysis on evading ransomware detection techniques,” Feb. 2022, doi: https://doi.org/10.1007/s00521-022-07096-6(.

[29] C. Pegoraro Chenet, A. Savino, and S. Carlo, “A survey on hardware-based malware detection approaches,” Apr. 2024. Accessed: Feb. 17, 2025. [Online]. Available: https://arxiv.org/pdf/2303.12525

[30] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, doi: https://doi.org/10.1038/nature14539.

[31] P. Baheti, “12 Types of Neural Networks Activation Functions: How to Choose?,” *www.v7labs.com*, Mar. 08, 2022. https://www.v7labs.com/blog/neural-networks-activation-functions (accessed Mar. 14, 2025).

[32] J. Brownlee, “A Gentle Introduction to the Rectified Linear Unit (ReLU) for Deep Learning Neural Networks,” *Machine Learning Mastery*, Apr. 20, 2019. https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/ (accessed Mar. 14, 2025).

[33] J. Brownlee, “How to Choose an Activation Function for Deep Learning,” *Machine Learning Mastery*, Jan. 17, 2021. https://machinelearningmastery.com/choose-an-activation-function-for-deep-learning/ (accessed Mar. 14, 2025).

[34] A. Subasi, “Machine learning techniques,” *Practical Machine Learning for Data Analysis Using Python*, pp. 177–179, 2020, doi: https://doi.org/10.1016/b978-0-12-821379-7.00003-5.

[35] N. McCullum, “Deep Learning Neural Networks Explained in Plain English,” *freeCodeCamp.org*, Jun. 28, 2020. https://www.freecodecamp.org/news/deep-learning-neural-networks-explained-in-plain-english/ (accessed Mar. 15, 2025).

[36] M. Sentner, “Forward Propagation in AI: Key Concepts Explained,” *Telnyx*, Mar. 08, 2025. https://telnyx.com/learn-ai/forward-propogation-ai (accessed Mar. 14, 2025).

[37] D. Bergmann and C. Stryker, “What is Loss Function? | IBM,” *www.ibm.com*, Jul. 12, 2024. https://www.ibm.com/think/topics/loss-function (accessed Mar. 15, 2025).

[38] D. Bergmann and C. Stryker, “What is Backpropagation? | IBM,” *Ibm.com*, Jul. 02, 2024. https://www.ibm.com/think/topics/backpropagation (accessed Mar. 15, 2025).

[39] Z. Keita, “An Introduction to Convolutional Neural Networks (CNNs),” *Datacamp.com*, Nov. 14, 2023. https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns (accessed Mar. 15, 2025).

[40] IBM, “Convolutional Neural Networks,” *Ibm.com*, Oct. 06, 2021. https://www.ibm.com/think/topics/convolutional-neural-networks (accessed Mar. 15, 2025).

[41] N. Srivastava, G. Hinton, A. Krizhevsky, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1931, 2014, Accessed: Mar. 15, 2025. [Online]. Available: https://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf

[42] GeeksforGeeks, “Introduction to Recurrent Neural Network - GeeksforGeeks,” *GeeksforGeeks*, Oct. 03, 2018. https://www.geeksforgeeks.org/introduction-to-recurrent-neural-network/ (accessed Mar. 15, 2025).

[43] GeeksforGeeks, “Deep Learning | Introduction to Long Short Term Memory,” *GeeksforGeeks*, Jan. 16, 2019. https://www.geeksforgeeks.org/deep-learning-introduction-to-long-short-term-memory/ (accessed Mar. 15, 2025).

[44] GeeksforGeeks, “Architecture and Working of Transformers in Deep Learning,” *GeeksforGeeks*, Jul. 29, 2024. https://www.geeksforgeeks.org/architecture-and-working-of-transformers-in-deep-learning/ (accessed Mar. 15, 2025).

[45] GeeksforGeeks, “Neural Architecture Search Algorithm,” *GeeksforGeeks*, Mar. 21, 2022. https://www.geeksforgeeks.org/neural-architecture-and-search-methods/ (accessed Mar. 16, 2025).

[46] L. Weng, “Neural Architecture Search,” *Github.io*, Aug. 06, 2020. https://lilianweng.github.io/posts/2020-08-06-nas/ (accessed Mar. 16, 2025).

[47] T. Elsken, Jan Hendrik Metzen, and F. Hutter, “Neural Architecture Search,” pp. 67–70, Jan. 2019, doi: https://doi.org/10.1007/978-3-030-05318-5_3.

[48] T. Elsken, J. Metzen, and F. Hutter, “Neural Architecture Search: A Survey,” *Journal of Machine Learning Research*, vol. 20, pp. 1–2, 2019, Accessed: Mar. 16, 2025. [Online]. Available: https://www.jmlr.org/papers/volume20/18-598/18-598.pdf

[49] omarmuhamed, “GitHub - omarmuhamed/Malware-Detection-With-Performance-Counters.,” *GitHub*, 2021. https://github.com/omarmuhamed/Malware-Detection-With-Performance-Counters/tree/main (accessed Mar. 16, 2025).

[50] VirusTotal, “VirusTotal,” *Virustotal.com*, 2000. https://www.virustotal.com/

[51] Amazon Web Services, “What Is Amazon EC2? - Amazon Elastic Compute Cloud,” *Amazon.com*, 2025. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html

[52] VirtualBox, “Introduction,” 2024, Available: https://userguide.m-files.com/user-guide/latest/eng/intro_to_m-files.html

[53] O. Mohamed and C.-B. Chirila, “Towards malware detection based on performance counters using deep learning classification models.” Accessed: Mar. 16, 2025. [Online]. Available: https://github.com/omarmuhamed/Malware-Detection-With-Performance-Counters/blob/main/paper/2022077090.pdf

[54] A. Alraizza and A. Algarni, “Ransomware Detection Using Machine Learning: A Survey,” *Big Data and Cognitive Computing*, vol. 7, no. 3, p. 8, Sep. 2023, doi: https://doi.org/10.3390/bdcc7030143.